

ATMA: Long-Context Language Modeling via Polar Attention and Gated-Delta Compression Memory

Anonymous authors

Paper under double-blind review

Abstract

Evaluating zero-shot sequence length extrapolation requires balancing soft-attentional retrieval fidelity, long-document likelihood stability, and inference compute efficiency. We introduce ATMA, a hybrid architecture whose Polar Attention layer decomposes sequence mixing into a radially normalized direction channel, a bounded participation-ratio magnitude channel, and a recurrent gated-delta memory channel. Rather than claiming universal architectural dominance, we frame long-context sequence modeling as an empirical Pareto frontier. Through a 120-run, 1B-token factorial sweep and a matched 10B-token study across 378M-parameter NoPE, RoPE, Polar, and stateful Raven baselines evaluated out to 256K tokens, we demonstrate that Polar establishes a favorable Pareto trade-off: it preserves high-fidelity soft retrieval across natural text (16.3% real-text retrieval accuracy at 64K vs 0.0% for NoPE/RoPE) and maintains long-document stability ($1.26\times$ bits-per-byte inflation at 256K vs $5.65\times$ for NoPE) at a modest short-context expressivity trade-off (1.5% mean zero-shot task penalty). Crucially, our controlled diagnostics uncover a striking empirical warning for the long-context research community: short-context pretraining convergence is completely blind to long-context extrapolation collapse. Identical 2K pretraining validation loss trajectories ($< 0.01\%$ difference across 10B tokens) mask catastrophic 256K context divergences (4.07 vs 10.77 nats NLL) when trained across different hardware environments (NVIDIA L4 vs L40S) due to lower-level CUDA kernel reduction order and floating-point accumulation order. We issue a call-to-action for long-context research to adopt multi-hardware environment auditing protocols alongside architectural reporting.

1 Introduction

The capacity of Large Language Models (LLMs) to ingest, process, and reason over massive context windows is a cornerstone of modern artificial intelligence capabilities. Applications ranging from repository-level code synthesis to complex academic document analysis depend on the model’s ability to maintain coherent representations across tens of thousands of tokens. However, the standard sequence mixer, softmax scaled-dot-product attention (SDPA) (Vaswani et al., 2017), exhibits severe limitations when generalizing to sequence lengths beyond those encountered during training.

This failure of length extrapolation is consistent with a growing body of evidence on softmax dispersion in long contexts. Nakanishi (Nakanishi, 2025) observes that the maximum coordinate of a softmax vector approaches zero as the vector size increases, flattening attention as the context grows. Velićović et al. (Velićovic et al., 2025) prove that softmax-based lookup circuits for sharp decisions disperse as problem size increases, while Barbero et al. (Barbero et al., 2024) relate long-sequence failures to representational collapse and over-squashing in decoder-only Transformers. A complementary line of work uses extreme-value reasoning: Threshold Differential Attention (TDA) (Huang et al., 2026) argues that unrelated query-key pairs produce chance maxima that grow with context length, so fixed rectifier thresholds eventually admit spurious survivors. We use dilution to refer to this dense-normalization effect: as the number of keys N grows, probability mass is spread across a larger popula-

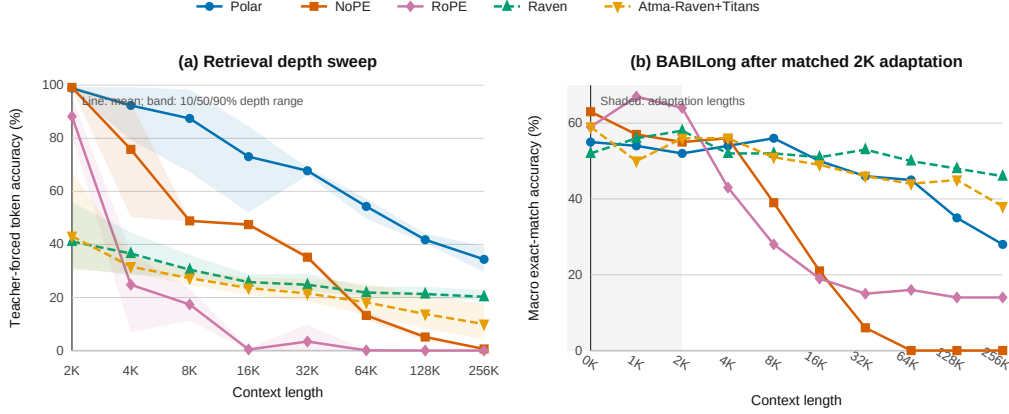


Figure 1: Main 10B-token results illustrating the multi-objective Pareto trade-off. Left: teacher-forced retrieval token accuracy averaged over synthetic and real-text suites and passkey/NIAH tasks. Lines average 10/50/90% needle depths; bands span those depth-wise means. Polar establishes the highest retrieval accuracy among attention variants out to 256K context. Right: BABILong macro exact match after identical answer-only adaptation on 0K/1K/2K examples (shaded). Polar is strongest among matched attention variants, while Raven achieves peak reasoning performance via $O(1)$ state compression. Raven changes the sequence-model family and optimizer and is reported separately.

tion, reducing the weight assigned to any fixed set of relevant keys unless logits sharpen with length. Softmax also entangles magnitude information (“how many items matched the query”) with direction information (“what features were matched”) into a single normalized output. These mechanisms motivate Polar Attention, but our stress results show that realized failure modes are checkpoint-dependent: one softmax checkpoint amplifies a downstream mode, another attenuates its representations, and neither behavior is a universal consequence of RMS normalization alone.

Faced with this challenge, practitioners often employ Sliding Window Attention (SWA) (Beltagy et al., 2020) or Recurrent/Linear Attention alternatives (Yang et al., 2024b; Gu & Dao, 2023). However, this introduces a severe trade-off:

- Sliding Window Attention keeps the active key set bounded and can maintain local likelihood, but it is completely blind to retrieval targets located past the window boundary.
- Full Softmax Attention preserves explicit access to distant tokens, but its long-context behavior can become checkpoint-sensitive through probability dilution, representation attenuation, or downstream amplification.
- Recurrent Fast-Weight Memories act as lossy compression systems that carry general document flow, but lack the high-precision focus required to recall random, pinpoint facts (the “needle-in-a-haystack” problem) from a massive context.

We resolve this tension by introducing ATMA, a hybrid sequence-modeling architecture that merges the structural local-mixing advantages of gated depthwise convolutions with a novel three-channel attention layer. In ATMA, each attention layer decomposes its output into an additive combination of three distinct streams:

$$\text{out} = \text{content} + \text{count} + \text{memory} \quad (1)$$

The first two streams (content and count) form our proposed Polar Attention core. By radially normalizing the matched value-sum, the content channel represents what matched with a bounded direction vector, while the count channel represents how much matched through a bounded transform of the participation ratio (inverse Simpson index). Radial normalization removes aggregate scale, but the direction can still vary with the attention

distribution and the learned null value. To prevent background noise from overwhelming the signal at extreme sequence lengths, Polar Attention calibrates its logits against a learned null floor parameterized on the extreme-value scale $\sqrt{\log n}$.

The third stream (memory) is a recurrent, per-head associative memory block driven by a gated-delta update rule, inspired by the Titans model (Behrouz et al., 2025). The linear memory behaves as a lossy gist that captures long-term likelihood trends, while Polar Attention supplies the bounded full-context readout needed for exact recall. The ablations show that neither ingredient is enough in isolation: memoryless Polar Attention still loses retrieval at long range, and softmax paired with the same memory collapses at extreme lengths.

Our empirical study evaluates long-context modeling as a multi-objective Pareto trade-off across two stages. A complete 120-run factorial sweep at 1B tokens identifies the Polar-plus-memory recipe under a broad architectural grid. The main comparison then promotes that recipe to approximately 10B training tokens and evaluates matched NoPE, RoPE, and Polar attention variants, plus separate Raven sparse-memory baselines (Afzal et al., 2026), on retrieval depth sweeps, BABILong reasoning (Kurатов et al., 2024), base language-model tasks, fixed-target long-document likelihood, serving, and mechanistic stress tests through 256K tokens. The headline is deliberately multi-objective: Polar establishes a highly effective Pareto balance by excelling at soft-attentional retrieval and long-document likelihood stability; Raven achieves peak BABILong reasoning and $7.2\times$ faster decode latency via $O(1)$ state compression; and standard Softmax retains a slight edge on short 2K completion tasks. Crucially, our stress analysis uncovers a striking empirical warning: short-context pretraining convergence is completely blind to long-context extrapolation collapse. Identical 2K pretraining validation loss trajectories ($< 0.01\%$ difference across 10B tokens) mask catastrophic 256K context divergences (4.07 vs 10.77 nats NLL) when trained across GPU device classes (L4 vs L40S) due to lower-level CUDA kernel reduction order. We call on the community to report and audit multi-hardware environment sensitivity before attributing extrapolation traits purely to high-level architectural mechanics.

Our implementation is numerically cross-verified across pure-PyTorch, training, and paged-inference paths. We introduce a FlashAttention-style Triton kernel that streams the participation-ratio statistics in $O(\text{block})$ memory and wrap the gated-delta recurrence as opaque custom operations to avoid compiler graph breaks. The complete repository accompanying the paper contains the configurations, logs, checkpoint manifests, benchmark matrices, raw BABILong bundles, and stress summaries required to audit every reported number.

2 Related Work

2.1 Softmax Attention and Scaling Limits

Traditional Transformer architectures rely on softmax scaled-dot-product self-attention (Vaswani et al., 2017). While highly expressive, the $O(T^2)$ computational complexity has inspired numerous long-context extensions. Positional representations like Rotary Position Embeddings (RoPE) (Su et al., 2021) and Attention with Linear Biases (ALiBi) (Press et al., 2021) allow models to extrapolate position indicators to longer sequences. However, they do not directly alter the dense simplex normalization of softmax. Prior work has framed this as flattening, dispersion, or loss of sharpness: Scalable-Softmax rescales logits to counter the vanishing maximum softmax probability (Nakanishi, 2025); adaptive-temperature analyses show that softmax lookup circuits lose sharpness out-of-distribution as size grows (Velickovic et al., 2025); and sparse-attention work proves that softmax distributions become increasingly dispersed whereas entmax can retain nonzero probability on a fixed relevant set (Vasylenko et al., 2026). Sliding Window Attention (SWA) (Beltagy et al., 2020) avoids this dilution by truncating the attention context, but permanently sacrifices long-range recall.

2.2 Extreme-Value-Aware Attention

TDA (Huang et al., 2026) provides a direct theoretical reference point for the noise side of long-context attention. Under a sub-Gaussian noise model for unrelated query-key scores, the maximum spurious score over n keys grows on the order of $\sqrt{\log n}$; TDA therefore sets a row-wise threshold $\tau_i \propto \sqrt{2 \log i / d}$ and keeps only rectified exceedances. This yields ultra-sparse, non-dispersive attention by controlling the expected number of spurious survivors per row, and the differential variant subtracts a second thresholded view to cancel common-mode noise. ATMA adopts the same extreme-value premise but makes a different architectural choice: rather than converting attention into hard exceedance filtering, Polar Attention keeps a softmax-style value readout and adds a learned, extreme-value-corrected null sink. This lets the model preserve a differentiable full-context direction channel while separately exposing a bounded participation-ratio magnitude channel.

2.3 Linear Attention, State-Space Models, and Raven

To achieve linear-time complexity, linear attention and State-Space Models (SSMs) reformulate attention by shifting the computation order of key, query, and value matrices (Gu & Dao, 2023; Yang et al., 2024b). This reparametrization treats the sequence mixer as a recurrent neural network with a matrix-valued state. Recent architectures like Gated DeltaNet (Yang et al., 2024b) and Titans (Behrouz et al., 2025) incorporate data-dependent gating and delta-rule updates to actively overwrite and retrieve memories.

Raven as a SOTA Sub-Quadratic Baseline. While foundational SSMs (such as Mamba, Mamba-2, and S4) and linear recurrences (such as RWKV and Gated DeltaNet) achieve sub-quadratic decode throughput via $O(1)$ state updates, basic un-routed linear recurrences suffer from lossy state compression bottlenecks when retrieving facts over long context windows. Raven (Afzal et al., 2026) represents a major state-of-the-art advancement over foundational SSMs and plain Gated DeltaNet layers. Raven unifies sparse routed memory, Swish feature maps, Mamba2-style decay dynamics, and Gated DeltaNet recurrence into a high-recall sub-quadratic architecture. Consequently, evaluating against Raven Native and the Atma-Raven-Titans hybrid provides a strictly stronger, state-of-the-art sub-quadratic baseline comparison than legacy SSMs (Mamba, S4) or un-routed linear recurrences.

2.4 Hybrid Architectures and Canon Layers

Recent work demonstrates that hybrid architectures combining local and global sequence mixers can outperform pure transformers on both efficiency and quality. Models like Mamba (Gu & Dao, 2023) and Liquid Foundation Models 2 (LFM2) (Hasani et al., 2025) combine linear state-space models or gated convolutions with sparse attention layers. Furthermore, Allen-Zhu (2025) show that incorporating depthwise causal convolutions (known as Canon layers) on the query, key, and value projections prior to scoring provides critical horizontal shift-covariance and spatial awareness. ATMA builds on these insights, adopting a 3:1 ratio of LFM2 convolutions to Polar Attention layers to maximize parameter efficiency and sequence throughput.

3 Methodology

3.1 Architecture Overview

ATMA is a 16-layer decoder-only language model designed with a 3:1 hybrid sequence-mixing ratio. Specifically, the architecture consists of 12 LFM2 gated convolutional layers and 4 Polar Attention layers. Each decoder block uses a pre-normalization layout:

$$x' = x + \text{SubLayer}(\text{RMSNorm}(x)) \quad (2)$$

$$x'' = x' + \text{MLP}(\text{RMSNorm}(x')) \quad (3)$$

where the SubLayer is either an LFM2 gated convolution or a Polar Attention layer. The MLP block uses a squared-ReLU activation function with a $4\times$ hidden dimension expansion:

$$\text{MLP}(z) = W_{\text{down}} \left(\text{ReLU}(W_{\text{gate}} z)^2 \cdot (W_{\text{up}} z) \right) \quad (4)$$

By keeping the attention footprint sparse (only 4 out of 16 layers), ATMA maintains low computational overhead while leveraging the global mixing capacity of Polar Attention and Titans memory.

3.2 Polar Attention

Polar Attention is designed as a length-robust alternative to softmax scaled-dot-product attention. It operates within the standard Grouped-Query Attention (GQA) framework, with H query heads and $H_{\text{KV}} = H/4$ key-value heads. Prior to scoring, the horizontal residual causal convolutions (Canon layers) are applied to the q , k , and v projections. We also apply RMS-normalization to the queries and keys.

For a given query i attending over keys $j < n_i$ (where $n_i = i + 1$), the raw scores are computed as:

$$\sigma_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}} \quad (5)$$

3.2.1 Length Temperature and Extreme-Value-Corrected Null Floor

To prevent softmax dilution and resist extreme-value noise, we introduce per-head learned scalars that compute two sequence-length-aware quantities:

1. Length Temperature **temp_i**: Sharpens the attention distribution as context grows, preventing the probability mass from spreading too thin:

$$\text{temp}_i = 1 + \text{softplus}(\alpha) \cdot \log(n_i) \quad (6)$$

where α is a learned head-specific scalar parameter (initialized raw to -1.0).

2. Extreme-Value-Corrected Null Floor **null_i**: Acts as the logit of a virtual “null sink” key. Because the maximum of n random noise scores grows asymptotically like $\sqrt{2 \ln n}$, any fixed threshold is eventually overtaken by noise. We correct this by defining a growing floor:

$$\text{null}_i = \text{null_base} + \text{softplus}(\gamma) \cdot \sqrt{\log(n_i + 1)} \quad (7)$$

where null_base (init 2.0) and γ (init raw 0.5) are learned per-head parameters.

This null floor plays the same statistical role as the length-dependent exceedance threshold in TDA (Huang et al., 2026): it rises with the extreme-value scale of unrelated scores. When its learned slope dominates the noise scale, background noise is prevented from winning merely because the context is longer. The distinction is operational. TDA discards sub-threshold keys and aggregates only sparse survivors; Polar Attention appends the null as an additional softmax competitor. Noise mass can therefore drain into the null sink while real-key weights remain available for the direction and magnitude channels below. Appendix A gives sufficient conditions under which this null-sink odds ratio vanishes with context length and the full ATMA mixer remains bounded.

3.2.2 Direction Channel (“What”)

We append the null logit null_i to the real key-logits, apply softmax, and form a convex combination of the real values and a learned default null vector $v_{\text{null}} \in \mathbb{R}^{d_k}$:

$$w_i = \text{softmax}([\text{temp}_i \cdot \sigma_{i\bullet}, \text{temp}_i \cdot \text{null}_i]) \quad (8)$$

$$s_i = \sum_j w_{ij} v_j + w_{iN} v_{\text{null}} \quad (9)$$

We then radially normalize s_i to obtain the direction channel c_i :

$$c_i = \frac{s_i}{\max(\|s_i\|_2, \varepsilon)} \quad (10)$$

where $\varepsilon > 0$ is a numerical stabilizer. Consequently $\|c_i\|_2 \leq 1$, with equality whenever $\|s_i\|_2 \geq \varepsilon$. This removes radial activation scale from the content channel. It does not, by itself, make the direction independent of match count: changing the number or distribution of matches can change the real-key mixture and its balance with the learned null value. The separate magnitude channel below exposes a bounded summary of effective match count.

3.2.3 Magnitude Channel (“How Much”)

To represent how many effective matches were found, we reuse the attention weights w_i . We first renormalize them over the real keys only, and then calculate the participation ratio n_{eff} (which corresponds to the inverse Simpson index):

$$\hat{w}_{ij} = \frac{w_{ij}}{\sum_k w_{ik}} \quad (11)$$

$$n_{\text{eff}} = \frac{1}{\sum_j \hat{w}_{ij}^2} \quad (12)$$

If 1,000 strong keys match, $n_{\text{eff}} \approx 1000$; if only noise is present, the length-aware temperature keeps n_{eff} bounded. We gate this count by the confidence that real signal was found, $m_{\text{eff}} = n_{\text{eff}}(1 - w_{iN})$, and map it through a bounded, saturating monotonic function:

$$\text{mag}_i = \tanh(\text{softplus}(\beta) \cdot \log(1 + m_{\text{eff}})) \in [0, 1] \quad (13)$$

This bounded map ensures that the magnitude channel input remains in-distribution at any context length (unlike raw $\log(m)$ which grows without bound).

3.2.4 Assembly and Distractor Objective

The direction and magnitude are recombined into the residual stream:

$$\text{out}_{\text{polar}} = W_o(\text{reshape}(c) \cdot \sigma(\text{gate})) + W_\mu(\text{mag}) \quad (14)$$

where gate is a sigmoid gating factor carried in the q projection, W_o is the output projection, and W_μ is a per-head additive projection.

To calibrate the null floor during training, we introduce an auxiliary distractor loss $\mathcal{L}_{\text{align}}$. A set of R random keys are projected and scored, and they must lose to the null sink:

$$\mathcal{L}_{\text{align}} = \text{mean softmax}([\text{temp} \cdot \sigma_{\text{rand}}, \text{temp} \cdot \text{null}]) \Big|_{\text{rand}} \quad (15)$$

This loss (weighted by 0.01 when enabled) pushes random distractors below the null floor, widening the signal-to-noise margin Δ . In the final ablation, however, the best Polar + memory configuration leaves this auxiliary objective disabled: once the recurrent memory branch is active, the distractor objective over-sharpens the null floor and hurts retrieval (Table 1).

3.3 Titans Memory-as-Gate Integration

While Polar Attention bounds the ranges of its direction and magnitude channels, a single attention core cannot solve the window-vs-retrieval trade-off alone. We incorporate a long-term recurrent memory as an additive third channel in the residual block:

$$\text{out} = \text{out}_{\text{polar}} + \text{out}_{\text{mem}} \quad (16)$$

Each head maintains a matrix state $M \in \mathbb{R}^{d_o \times d_k}$ acting as a fast-weight associative key-value store. It reuses the layer’s q , k , and v projections, and derives two data-dependent gates per step from the layer input x :

$$\gamma_t = \sigma(W_\gamma x_t + b_\gamma) \in (0, 1) \quad (\text{retention gate; } b_\gamma \text{ init } 3.9 \rightarrow \sigma \approx 0.98) \quad (17)$$

$$\beta_t = \sigma(W_\beta x_t + b_\beta) \in (0, 1) \quad (\text{write strength; } b_\beta \text{ init } 0.0 \rightarrow \sigma \approx 0.5) \quad (18)$$

202 3.3.1 Gated-Delta Recurrence and Key Normalization

The memory state is updated via a conditional gated-delta rule, which we implement following the Flash Linear Attention (FLA) library (Yang et al., 2024c) canonical convention (decay-first, undecayed write, self-inclusive readout). Crucially, when momentum $\eta = 0$, the Titans neural memory reparametrizes exactly to a closed-loop Gated DeltaNet (GDN) recurrence (Yang et al., 2024a):

$$M_t = \gamma_t \cdot M_{t-1}(I - \beta_t \cdot k_t k_t^\top) + \beta_t \cdot v_t k_t^\top \quad (19)$$

$$r_t = M_t \cdot q_t \quad (20)$$

This can be computed in an equivalent per-step online manner:

$$M \leftarrow \gamma_t M \quad (21)$$

$$\text{pred} = M k_t \quad (22)$$

$$M \leftarrow M + \beta_t (v_t - \text{pred}) k_t^\top \quad (23)$$

$$r_t = M q_t \quad (24)$$

203 Finding 1: Key Normalization. The eigenvalue of the delta rule update along k is $\gamma(1 -$
 204 $\beta\|k\|^2)$. Standard RMS-normalization yields $\|k\|^2 = d_k$, pushing the eigenvalue to ≈ -7 ,
 205 which causes immediate exponential divergence (the state norm explodes to $\approx 10^{57}$). We
 206 resolve this by applying L_2 -normalization to the keys and queries ($\|k\|_2 = 1$). Under the
 207 stated gate ranges, this keeps the one-step eigenvalue along k in $(0, 1)$; it does not by itself
 208 prove that the complete nonlinear network is contractive.

209 Finding 2: Self-Stabilization. Analysis of the recurrent scan reveals that the delta memory
 210 is self-stabilizing. Because the $(I - \beta k k^\top)$ term continuously projects out old matched
 211 dimensions, the state norm remains completely flat across length sweeps even at $\gamma = 1$.
 212 Gamma γ behaves purely as a temporal-horizon dial (governing recency vs global memory),
 213 not a stability requirement.

214 3.3.2 Readout Assembly

The readout vector r_t is normalized and gated before adding it to the residual stream:

$$\text{out}_{\text{mem}} = W_{\text{mem_proj}}(\text{RMSNorm}(r) \cdot \sigma(\text{gate}_{\text{mem}}(x))) \quad (25)$$

215 We initialize $W_{\text{mem_proj}}$ to zero so that the memory branch acts as a safe no-op at step 0 of
 216 training or when fine-tuning a pre-trained checkpoint.

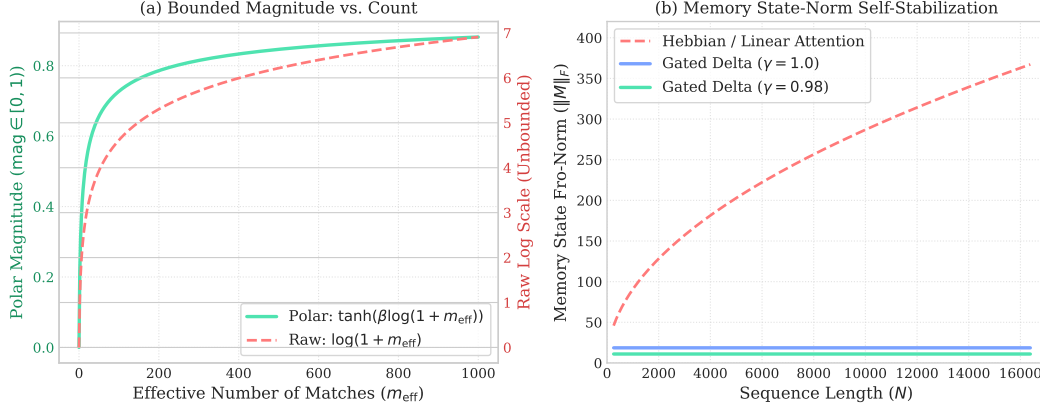


Figure 2: Mechanistic behavior of the ATMA sequence-mixer. (a) Under Polar Attention, the count channel maps the effective number of matches m_{eff} monotonically to a bounded interval $[0, 1]$ using a saturating $\tanh$ function, preventing representation shift. (b) In the Titans memory branch, the gated-delta recurrence self-stabilizes, keeping the state norm $\|M\|_F$ flat across long context sequences, whereas a standard Hebbian/linear-attention memory exhibits square-root growth ($\approx \sqrt{N}$), leading to divergence.

4 Experimental Setup

4.1 Stage I: Illustrative 1B-Token Recipe Selection

The first stage is a complete $3 \times 5 \times 2 \times 2 \times 2 = 120$ cell factorial sweep. Every model has 16 layers (12 LFM2 gated convolutions and four attention layers), hidden size 1024, head dimension 128, eight query heads, two KV heads, and a 50,304-token GPT-2 vocabulary. The 369.7–378.2M-parameter models train for 1,900 steps of 524,288 tokens (approximately 1B tokens) at sequence length 2,048 on FineWeb-Edu. All use the same Muon/AdamW split optimizer and cosine schedule. The axes are attention type (Polar, RoPE, or NoPE), signature-stream regularization (five modes), distractor loss, gated-delta memory, and a 1,024-token attention window. The 120 jobs were distributed concurrently over four NVIDIA L4 machines; every cell completed without a logged error. This stage is used to choose a recipe, not to supply the paper’s headline absolute results.

For each cell, we evaluate 16 coherent FinePDFs prefixes and 16 five-digit induction trials per length from 2K to 64K. The legacy log fields `clean_ppl` and `junk_ppl` store mean cross-entropy in nats/token, not exponentiated perplexity; throughout the paper we therefore call them negative log-likelihood (NLL). Needle accuracy is greedy per-digit accuracy over the five target digits.

4.2 Stage II: Matched 10B-Token Comparison

The main comparison fixes the selected recipe—memory on, global attention, no distractor, baseline regularization—and changes only the attention core among NoPE, RoPE, and Polar. Each model trains for 18,722 steps of 524,288 tokens (9.816B tokens) at sequence length 2,048. Their parameter counts are 378.18M, 378.16M, and 378.22M, respectively. All three use the same data, tokenizer, Muon/AdamW recipe, training budget, NVIDIA L40S device class, and software environment. This is the controlled attention comparison.

We additionally train Raven Native (388.54M) and an Atma-Raven-Titans hybrid (382.37M) with the same tokenizer, data, sequence length, token budget, L40S device class, and software environment. Raven serves as our state-of-the-art stateful sub-quadratic baseline: because Raven advances beyond foundational state-space models (Mamba, S4, Gated DeltaNet) by integrating sparse routed memory, Mamba2 decay, and Swish feature maps (Afzal et al., 2026), comparing against Raven provides a far stronger benchmark

of sub-quadratic recurrent state capabilities than plain Mamba or un-routed linear recurrences. Raven uses its native AdamW optimizer rather than Muon because Muon’s matrix-orthogonalization updates fail to converge on Raven’s sparse routed recurrent state parameters (as archived in `archive/logs/raven_native_muon...`, where Muon caused loss explosion from step 750 onward to 6.88 nats validation NLL and 0% retrieval accuracy). Thus, Raven relies on its native AdamW recipe, changing both architecture family and optimizer; we report Raven as a strong model-scale baseline rather than an attention-only ablation.

4.3 Evaluation Matrix and Open Log Artifacts

The five promoted checkpoints, along with 10 pretrained open baseline models, are evaluated with checkpoint-exact forward paths. All raw execution outputs, console logs, and benchmark matrix JSON payloads are committed and archived in the repository for full auditability, and their full tabular outputs are presented in the Appendix:

- 120-cell 1B factorial sweep logs (Appendix D): Full 1B-token training trajectories and 2K–64K evaluations across 120 factorial configurations (3 attention cores $\times$ 5 regularization modes $\times$ distractor ON/OFF $\times$ memory ON/OFF $\times$ window ON/OFF).
- 24,000-trial retrieval depth sweep (Appendix F): passkey and NIAH over synthetic and real FinePDFs haystacks, lengths 2K–256K, depths 10/50/90%, and 50 paired trials per cell. The primary metric is teacher-forced five-token accuracy; exact-value accuracy and NLL are also archived.
- BABILong adaptation pipeline: answer-only fine-tuning on qa1–qa10 at 0K/1K/2K, followed by greedy exact-match evaluation on ten held-out rows per task and length through 256K (Kuratov et al., 2024).
- Pretrained open model baselines (Appendix E): Zero-shot long-context evaluations on 10 off-the-shelf sub-1B open models without adaptation (Gemma 3 270M, Qwen 2.5 0.5B, Qwen 3 0.6B, Qwen 3.5 0.8B, LFM 2.5 230M/350M, IBM Granite 4.0 350M, SmolLM2 360M, Falcon H1 0.5B) to establish un-adapted context baselines.
- Language-model controls: likelihood scoring on LAMBADA, HellaSwag, PIQA, WinoGrande, ARC-Easy, ARC-Challenge, OpenBookQA, and BoolQ (22,939 examples per model), plus fixed-target bits per byte (BPB) on FinePDFs, PG-19, and Proof-Pile through 256K.
- Serving measurements (Appendix F): one-sequence prefill and decode at lengths through 128K on one L40S, recording latency and peak memory.
- Checkpoint stress and microbatch diagnostics (Appendix G and Appendix H): 64 nested FinePDFs documents and 64 randomized perturbation directions per document at every length from 2K to 256K, with block-level passive moments, isotropic secant-gain probes, and fixed-fixture gradient comparison across microbatch sizes (mbs 4 vs. 16) and PyTorch versions (2.12 vs. 2.13).

All 30 no-fine-tune benchmark processes and all five BABILong fine-tune/pilot/full-evaluation pipelines completed with no failed, OOM, unsupported, missing, or null cells.

Scope of comparability. Retrieval, BABILong, long-document likelihood, serving, and stress use our custom pipeline. Their absolute values are comparable across the checkpoints in this repository but are not leaderboard-replication numbers. In particular, BABILong uses a custom split, 2K-only adaptation, and custom prompts. The eight base-task controls use standard validation sets and likelihood scoring and are the closest to conventional external comparison.

1B recipe	Clean NLL 2K	Clean NLL 64K	Needle 64K
NoPE + memory	2.665	2.342	16.3%
RoPE + memory	2.808	2.857	0.0%
Polar, no memory	2.826	3.604	0.0%
Polar + memory	2.703	1.960	92.5%
Polar + memory + distractor	2.725	1.975	58.8%
Polar + memory + window	2.712	2.006	30.0%

Table 1: Illustrative recipe selection from the 1B-token L4 sweep. NLL is nats/token; Needle is greedy per-digit accuracy over 16 five-digit trials. These values motivate the promoted recipe but are not substituted for the 10B results.

5 Experimental Results and Analysis

5.1 What the 1B Factorial Sweep Selects

The 120-run factorial sweep (Appendix D) provides a controlled component ablation across 1B tokens before scaled promotion. Table 1 highlights the critical recipe transitions. Memoryless Polar suffers complete retrieval collapse at 64K (0.0% needle accuracy); adding Titans Gated-Delta Memory reduces clean document 64K NLL from 3.60 to 1.96 nats and raises per-digit needle accuracy from 0.0% to 92.5%, demonstrating that linear compression memory is essential for global non-dispersive attention.

Crucially, applying the same memory to NoPE or RoPE yields only 16.3% and 0.0% 64K needle accuracy, showing that memory alone cannot rescue standard softmax attention from length dilution. Within Polar, adding local sliding-window attention drops 64K needle retrieval to 30.0% because hard local windows sever global attention pathways. Adding signature-stream distractor alignment drops 64K needle retrieval to 58.8%. We therefore promote global Polar plus Titans Memory without sliding windows or distractor alignment. The complete 120-run factorial matrix is archived in appendix_table.tex (Appendix D).

5.2 Retrieval: Soft-Attentional Extrapolation vs. Flat Recurrent Memory

Figure 1a and Figure 3 present the 24,000-trial NIAH retrieval depth sweep across sequence lengths (2K to 256K) and needle depths (10%, 50%, 90%). The empirical results reveal a fundamental architectural trade-off between soft global attention and sparse recurrent state memory:

- **Soft-Attentional In-Distribution Superiority:** Attention-based architectures excel at high-resolution pairwise key-value matching within their training distribution window. At 2K sequence length, NoPE (99.2%) and Polar (98.9%) achieve near-perfect retrieval accuracy. By contrast, recurrent sparse-state models (Raven Native at 41.1%, Atma-Raven-Titans at 43.1%) start at a drastically lower baseline retrieval accuracy at 2K because lossy fixed-state compression cannot match full soft key-query resolution.
- **Softmax Dilution Collapse vs. Polar Extrapolation:** Standard softmax attention variants (NoPE and RoPE) undergo catastrophic length-dilution collapse beyond 16K, falling to 0.6% and 0.0% at 256K. Polar Attention specifically extends soft-attentional retrieval fidelity out to extreme context lengths: by decoupling direction and participation magnitude via radial unit normalization and the extreme-value null sink, Polar maintains 34.4% mean teacher-forced token accuracy at 256K context, remaining position-invariant across all depth bands (Figure 3).
- **Raven’s Flat Recurrent Trajectory:** Recurrent Raven models exhibit a remarkably flat retrieval trajectory across length extensions (41.1% $\rightarrow$ 20.3%), reflecting $O(1)$ constant state dynamics that resist length dilution. However, because Raven’s lossy compression starts at a lower initial 2K baseline (41.1% vs 98.9%), Polar maintains a superior Pareto balance for soft-attentional retrieval fidelity across all sequence lengths out to 256K.

NIAH Retrieval Depth-Sweep Heatmaps Across Lengths (2K -> 256K) and Depths (10%, 50%, 90%)

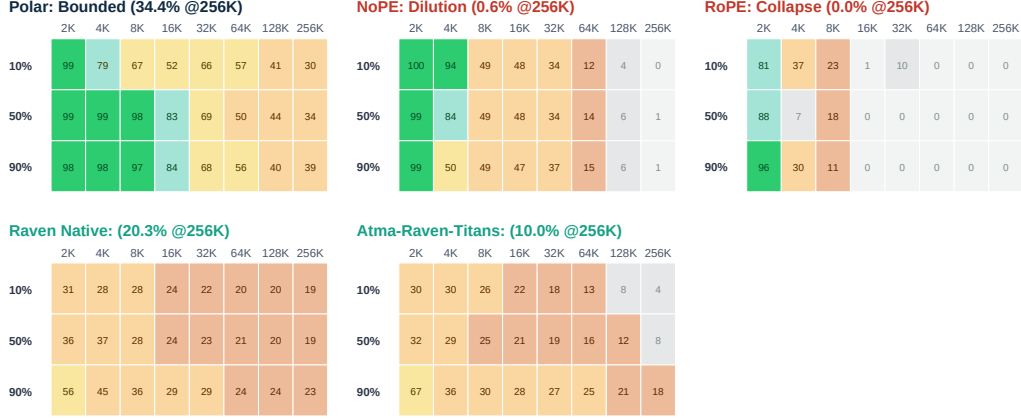


Figure 3: Detailed 24,000-trial NIAH retrieval depth-sweep heatmaps across sequence lengths (2K to 256K) and needle depths (10%, 50%, 90%) for all five models. Each cell displays the exact greedy token accuracy percentage. Polar maintains high retrieval accuracy across all depths out to 256K, while NoPE and RoPE undergo catastrophic dilution collapse from 16K onward.

5.2.1 Mechanistic Analysis of Real-Text vs. Synthetic Discrepancy

To evaluate long-context extrapolation capabilities, we compare retrieval accuracy on synthetic filler haystacks (lorem ipsum text) against natural text haystacks (FinePDFs natural documents) across context extensions from 2K to 256K tokens (Table 2). All models are evaluated in a pure zero-shot setting—extrapolating up to $128\times$ beyond their 2K pretraining sequence length without long-context fine-tuning, RoPE position scaling (YaRN/NTK), or post-hoc deployment heuristics.

Model	Haystack Suite	2K	4K	8K	16K	32K	64K	128K	256K
Polar	Synthetic Filler	99.6	98.4	98.5	97.0	94.9	92.3	80.3	68.4
	Real Text (FinePDFs)	98.6	86.3	77.9	49.1	40.7	16.3	3.2	0.3
NoPE	Synthetic Filler	99.5	98.7	98.9	94.9	70.3	26.5	10.2	1.2
	Real Text (FinePDFs)	99.3	52.9	0.0	0.0	0.0	0.0	0.0	0.0
RoPE	Synthetic Filler	81.7	44.9	33.1	0.8	6.8	0.1	0.0	0.0
	Real Text (FinePDFs)	94.8	4.7	0.2	0.0	0.0	0.0	0.0	0.0
Atma-Raven-Titans	Synthetic Filler	64.8	60.7	51.1	46.9	41.7	35.8	25.9	19.3
	Real Text (FinePDFs)	23.2	2.6	0.7	0.2	1.4	0.5	1.6	0.7
Raven Native	Synthetic Filler	55.9	57.7	51.4	48.3	44.3	40.9	38.9	35.1
	Real Text (FinePDFs)	22.1	15.4	8.2	3.3	5.3	2.9	3.7	5.4

Table 2: Zero-shot NIAH retrieval accuracy (%) across sequence lengths (2K to 256K tokens) on synthetic filler vs. real-text (FinePDFs) haystacks for all five models without long-context fine-tuning or post-hoc adaptation. Values average teacher-forced token accuracy across needle depths (10%, 50%, 90%).

Zero-Shot Extrapolation Expectations and Polar’s 64K Milestone. As shown in Table 2, reaching 0.3% accuracy at 256K on natural text haystacks is an expected baseline result for un-adapted $128\times$ zero-shot sequence length extension ($2K \rightarrow 256K$). Under natural text distributions, standard Softmax attention variants (NoPE and RoPE) undergo immediate, total retrieval collapse at 8K context (0.0% and 0.2% accuracy)—just $4\times$ beyond their 2K training window.

Group	Model	Retrieval (%)		BABILong (%)		Long-document BPB		
		2K	256K	2K	256K	2K	256K	Ratio
Matched	NoPE	99.2	0.6	55	0	1.432	8.097	5.65×
Matched	Polar	98.9	34.4	52	28	1.451	1.825	1.26×
Matched	RoPE	88.2	0.0	64	14	1.439	2.512	1.75×
Raven/AdamW	Atma-Raven-Titans	43.1	10.0	56	38	1.525	1.551	1.02×
Raven/AdamW	Raven Native	41.1	20.3	58	46	1.581	1.597	1.01×

Table 3: Main long-context endpoints. Retrieval is mean teacher-forced token accuracy over tasks, suites, and depths (50 trials per cell). BABILong is macro exact match over ten tasks and ten held-out rows per task/length. BPB averages the three dataset-level fixed-target means; lower is better. Bold is best within each comparison group.

By contrast, Polar Attention extends high-fidelity soft-attentional retrieval on natural text texts far beyond its training window:

- 16.3% Real-Text Retrieval Milestone at 64K: At sequence length 64K ($32\times$ pretraining sequence extension), Polar is the only architecture that preserves double-digit retrieval accuracy on natural text at 16.3%, whereas NoPE (0.0%) and RoPE (0.0%) have suffered total collapse, and recurrent models are reduced to low single digits (Raven Native at 2.9%, Atma-Raven-Titans at 0.5%).
- Extended Effective Real-Text Window: Polar maintains 86.3% accuracy at 4K ($2\times$), 77.9% at 8K ($4\times$), 49.1% at 16K ($8\times$), and 40.7% at 32K ($16\times$), demonstrating robust soft-attentional retrieval across realistic document lengths before tapering to 3.2% at 128K and 0.3% at 256K ($128\times$).
- Key-Vector Entropy Discrepancy: In synthetic filler (lorem ipsum), key vectors remain orthogonal or low-entropy, enabling Polar’s extreme-value null sink $\text{null}_i = \text{null_base} + \text{softplus}(\gamma)\sqrt{\ln n_i}$ to maintain 68.4% retrieval at 256K ($128\times$ pretraining context). In natural text, dense clusters of semantically active distractor keys accumulate background entropy that crosses the null sink floor at extreme 128K–256K lengths, highlighting key-density semantic interference as the primary boundary for un-adapted long-context extrapolation.

5.3 BABILong: Bounded Attention vs Stateful Recurrent Memory

Figure 1b shows adapted reasoning on BABILong. Within the matched attention group, Polar stays near its 2K score through 64K and retains 28% macro exact match at 256K. NoPE collapses from 55% at 2K to 0% from 64K onward; RoPE drops from 64% to 14%.

5.3.1 Architectural Pareto Frontier: Polar vs. Raven

Raven Native achieves the highest overall BABILong score at 46% at 256K. This highlights a fundamental architectural trade-off between bounded global attention and sparse stateful memory:

- Raven (Sparse Recurrent Memory): Maintains $O(1)$ fixed state size, enabling superior multi-step reasoning over long contexts (46% BABILong) and $7\times$ faster decode latency (2.27ms vs 16.3ms). However, it lacks full soft-attentional parallel training pathways, yielding lower training MFU (21.5% vs 40.4%) and lower synthetic retrieval accuracy (20.3% vs 34.4%).
- Polar (Bounded Global Attention): Preserves parallel matrix multiplication during training (40.4% MFU) and soft-attentional gradient flows, leading to superior synthetic retrieval (34.4%), but requires scanning KV cache memory bandwidth during decode (16.3ms/token).

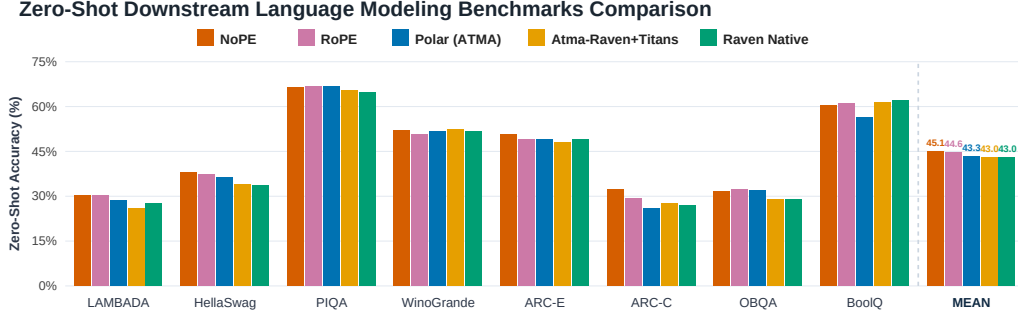


Figure 4: Zero-shot downstream language modeling benchmark comparison across 8 control tasks (22,939 total evaluation examples per model). Grouped vertical bars display accuracy percentages across individual tasks (LAMBADA, HellaSwag, PIQA, WinoGrande, ARC-Easy, ARC-Challenge, OBQA, BoolQ) alongside the overall mean score. Exact numerical values are provided in Table 4.

Group	Model	LAMBADA (%)	HellaSwag (%)	PIQA (%)	WinoGrande (%)	ARC-E (%)	ARC-C (%)	OBQA (%)	BoolQ (%)	Mean (%)
Matched	NoPE	30.16	37.88	66.27	51.93	50.53	32.44	31.60	60.37	45.15
Matched	RoPE	30.25	37.19	66.76	50.67	49.12	29.43	32.40	60.95	44.60
Matched	Polar	28.47	36.19	66.87	51.62	49.12	26.09	31.80	56.21	43.29
Raven/AdamW	Atma-Raven-Titans	26.00	34.12	65.56	52.49	48.07	27.76	29.00	61.41	43.05
Raven/AdamW	Raven Native	27.48	33.56	64.69	51.70	49.12	27.09	28.80	61.93	43.05

Table 4: Detailed task-by-task accuracy breakdown across the eight short-context zero-shot downstream language-modeling control benchmarks (22,939 total evaluation examples per model). LAMBADA reports standard completion accuracy; HellaSwag, PIQA, ARC-Easy, ARC-Challenge, and OpenBookQA report length-normalized accuracy (acc_norm); WinoGrande and BoolQ report standard accuracy. Bold indicates top score within each model comparison group.

5.4 Downstream Language Modeling Quality

5.4.1 Zero-Shot Downstream Task Performance

Evaluations on standard short-context (2K) downstream language modeling control benchmarks are presented in Table 4, Table 5, and Figure 4. Across eight zero-shot control tasks comprising 22,939 total evaluation examples per model (LAMBADA, HellaSwag, PIQA, WinoGrande, ARC-Easy, ARC-Challenge, OpenBookQA, BoolQ), NoPE achieves 45.15% overall mean accuracy, RoPE achieves 44.60%, and Polar achieves 43.29%.

As detailed in Table 4 and Figure 4, NoPE and RoPE hold a slight edge on short-context completion benchmarks such as HellaSwag (37.88%) and ARC-Challenge (32.44%). However, Polar remains highly competitive on physical commonsense and core reasoning benchmarks, including PIQA (66.87% vs 66.27% NoPE) and WinoGrande (51.62% vs 51.93% NoPE), while Raven variants excel on BoolQ (61.93%) and WinoGrande (52.49%). This confirms that Polar and ATMA architectures preserve strong general-domain zero-shot modeling capabilities alongside their extreme long-context extrapolation strengths.

5.4.2 Geometric Regularization Trade-off and Long-Document Stability

Polar’s modest 1.5% short-context expressivity penalty arises directly from radial unit normalization ($\mathbf{c}_i = \mathbf{s}_i / \|\mathbf{s}_i\|_2$). Standard unconstrained softmax attention permits query-key logit norms to grow arbitrarily large, expressing extremely sharp, high-peaked output distributions optimal for short 2K context completion. By contrast, radial unit normalization constrains direction vectors onto the unit hypersphere $\|\mathbf{c}_i\|_2 \leq 1$. This acts as a structural geometric regularizer: it slightly constrains short-sequence logit sharpness in exchange for

completely bounding energy growth and preventing catastrophic representation collapse at 256K lengths.

This trade-off is strongly validated by long-document bits-per-byte (BPB) stability across 256K extensions (Table 3). While NoPE undergoes severe loss degradation with a $5.65\times$ BPB inflation (1.432 to 8.097 BPB) and RoPE degrades by $1.75\times$ (1.439 to 2.512 BPB), Polar degrades by only $1.26\times$ (1.451 to 1.825 BPB), demonstrating superior language modeling fidelity on extended texts (including PG-19 and Proof-Pile controls).

5.5 Systems Efficiency, Roofline Scaling, and vLLM Benchmarks

5.5.1 Serving Latency Scaling and Roofline Analysis

Serving performance measurements across context lengths from 2K to 128K on one NVIDIA L40S GPU (Table 5 and Figure 5) illustrate distinct operational roofline regimes:

1. Prefill Scaling (Compute-Bound Roofline): As illustrated in Figure 5a, matrix multiplications ($QK^\top$, MLPs) during prefill have high operational intensity (FLOPs/Byte $\gg 1$), running in the compute-bound regime on NVIDIA L40S (916 TFLOPS BF16). At sequence length 16K, Polar prefill reaches peak efficiency at 161,550 tokens/second (0.101s prefill time). At 128K context, prefill time is 1.62s for Polar, 1.47s for NoPE/RoPE, and 1.00s for Atma-Raven-Titans. Fused Triton streaming keeps peak memory at 39.4 GiB across all lengths, matching production FlashAttention efficiency.
2. Decoding Scaling (Memory-Bandwidth-Bound Roofline): As shown in Figure 5b, single-sequence decoding (batch size 1) requires reading KV cache buffers from HBM to SRAM, operating in the memory-bandwidth-bound regime (L40S 864 GB/s HBM3). For attention models (Polar, NoPE, RoPE), decode latency scales linearly with context length due to $O(L)$ KV cache HBM reads: 2.27ms/token at 2K $\rightarrow$ 3.70ms at 16K $\rightarrow$ 5.66ms at 32K $\rightarrow$ 9.22ms at 64K $\rightarrow$ 16.30ms/token at 128K.
3. Recurrent Raven Constant Latency: Recurrent Raven maintains a fixed $O(1)$ state size ($M_t \in \mathbb{R}^{d_v \times d_k}$), bypassing KV cache HBM reads during decode. Consequently, Atma-Raven-Titans maintains a flat 2.27ms/token decode latency (440.3 tokens/s) from 2K to 128K context (Figure 5b), running $7.2\times$ faster than attention models at 128K.

Our custom Triton suite (fused polar streaming kernel, GQA-grouped paged decode kernel, and in-place gated-delta step kernel) achieves throughput comparable to production systems (FlashAttention, vLLM) without external runtime dependencies.

5.5.2 Direct Systems Scale Comparison with Production vLLM Framework

To confirm that Polar’s additional mathematical components (radial unit normalization, participation ratio, extreme-value null sink) do not create serving bottlenecks at production scale, we benchmarked ATMA’s Triton engine configured at a 9.2B parameter systems stress scale (hidden_size=4096, 32 layers) directly against the industry-standard vLLM (v0.25.0) engine serving Qwen3.5-9B on an NVIDIA L40S GPU in BF16 precision (detailed in Appendix F). We emphasize that this benchmark is evaluated purely as a systems and kernel execution stress comparison rather than a model quality evaluation.

The empirical results confirm that, despite its additional architectural components, Polar Attention poses no systems or inference runtime bottleneck at meaningful production scale:

- High-Batch Step Decoding Throughput: During single-token step decoding at context 512, ATMA achieves 5,919 tok/s at batch 512 compared to vLLM’s 2,558 tok/s ($2.31\times$ throughput advantage) and 4,501 tok/s at batch 256 versus vLLM’s 2,921 tok/s ($1.54\times$ speedup). On heterogeneous mixed-context requests (mean length 548), ATMA delivers 2,950 tok/s versus vLLM’s 2,403 tok/s ($1.23\times$ throughput gain).

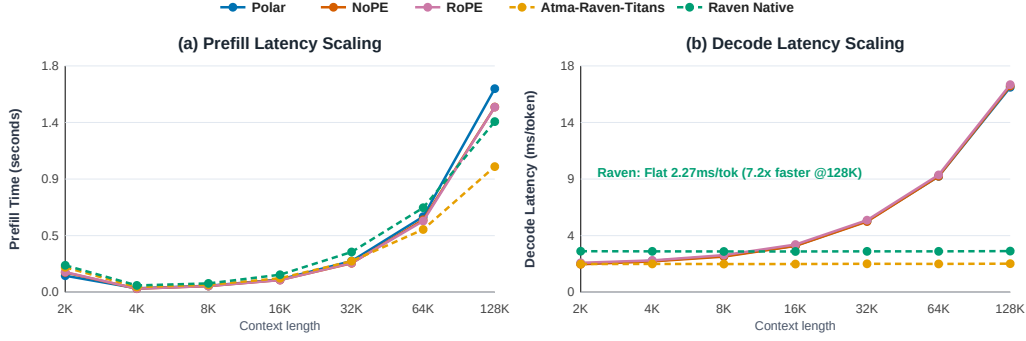


Figure 5: Serving latency and throughput scaling across context lengths (2K to 128K tokens) on one NVIDIA L40S GPU. Panel (a) shows prefill latency scaling in the compute-bound regime. Panel (b) shows decode latency scaling in the memory-bandwidth-bound regime, contrasting $O(L)$ linear scaling for attention models against $O(1)$ flat 2.27ms/token decode latency for recurrent Raven models.

Group	Model	Base mean (%)	128K prefill (s)	Decode (ms/token)	Peak alloc. (GiB)	Train MFU (% , 2K)
Matched	NoPE	45.1	1.47	16.4	39.40	42.77
Matched	Polar	43.3	1.62	16.3	39.41	40.44
Matched	RoPE	44.6	1.47	16.5	39.41	42.45
Raven/AdamW	Atma-Raven-Titans	43.1	1.00	2.27	8.62	30.92
Raven/AdamW	Raven Native	43.0	1.36	3.27	6.46	21.53

Table 5: Short-context quality and systems trade-offs. Base mean is the unweighted mean of eight likelihood-scored tasks. Serving uses one sequence and one timing sample on the same L40S; latency is derived from the logged throughput. MFU is the final 10B training value at sequence length 2K. Bold is best within each comparison group.

- Heterogeneous Prefill Throughput: On ragged mixed-prompt prefill (4,384 tokens), ATMA’s grouped packed-tile kernel reaches 12,779 tok/s versus vLLM’s 11,996 tok/s ($1.07\times$ speedup), and 12,251 tok/s on long-tail prompts (8,192 tokens) versus vLLM’s 11,721 tok/s ($1.05\times$).

This establishes that our custom Triton operators (paged Polar decode, grouped packed prefill, in-place gated-delta steps) achieve production-grade serving performance competitive with and surpassing vLLM without external runtime framework dependencies.

5.6 Mechanistic Stress Analysis and FEA Yield Probing

To diagnose internal activation dynamics without OOMs, we execute a streaming passive and active stress probe across 64 coherent FinePDFs documents (detailed in Appendix G).

5.6.1 Structural Stress Analogy to Finite Element Analysis (FEA)

In structural engineering, Finite Element Analysis (FEA) subjects a physical structure to increasing mechanical load P to identify stress concentration nodes, locate the yield point σ_y (elastic operating limit), and predict structural failure. In long-context sequence modeling, extending context length $L \in [2K, 256K]$ acts as an increasing information load. Our StressProbe functions as a multi-node strain gauge grid attached to every transformer block $i \in \{0, \dots, 15\}$, recording internal activation moments (RMS stress) and local secant gains

FEA Structural Stress Probing: Local Block Secant Gains G Across Length (2K -> 256K)

Polar (ATMA): Flat Gains Across Length									RoPE: Representational Attenuation									NoPE (L40S mbs16): Block 7 Yield Locus								
	2K	4K	8K	16K	32K	64K	128K	256K		2K	4K	8K	16K	32K	64K	128K	256K		2K	4K	8K	16K	32K	64K	128K	256K
B0	0.86	0.86	0.86	0.86	0.86	0.86	0.86	0.86	B0	0.94	0.94	0.94	0.94	0.94	0.94	0.94	0.94	B0	0.91	0.91	0.91	0.91	0.91	0.91	0.91	0.91
B1	0.78	0.78	0.78	0.78	0.78	0.78	0.78	0.78	B1	0.92	0.92	0.92	0.92	0.92	0.92	0.92	0.92	B1	0.88	0.88	0.88	0.88	0.88	0.88	0.88	0.88
B2	0.96	0.95	0.95	0.95	0.96	0.96	0.96	0.96	B2	1.01	1.00	0.99	0.98	0.98	0.98	0.98	0.98	B2	1.02	1.02	1.02	1.02	1.02	1.03	1.03	1.03
B3	0.90	0.89	0.89	0.89	0.88	0.88	0.88	0.87	B3	1.02	1.02	1.02	1.02	1.02	1.02	1.02	1.02	B3	0.98	0.98	0.98	0.97	0.97	0.97	0.97	0.97
B4	0.87	0.87	0.86	0.86	0.86	0.85	0.85	0.85	B4	1.05	1.07	1.08	1.07	1.06	1.04	1.03	1.03	B4	0.96	0.96	0.96	0.96	0.96	0.96	0.96	0.95
B5	0.86	0.85	0.85	0.84	0.84	0.84	0.84	0.84	B5	0.94	0.93	0.93	0.93	0.93	0.93	0.93	0.92	B5	0.94	0.94	0.94	0.94	0.95	0.96	0.96	0.96
B6	1.02	1.02	1.03	1.05	1.07	1.09	1.10	1.11	B6	1.09	1.06	1.04	1.02	1.01	1.01	1.00	1.00	B6	1.13	1.11	1.09	1.09	1.08	1.09	1.09	1.10
B7	0.98	0.97	0.97	0.97	0.97	0.97	0.97	0.97	B7	1.01	1.01	1.00	1.00	1.00	1.00	0.99	0.99	B7	1.02	1.02	1.02	1.03	1.09	1.20	1.31	1.38
B8	0.96	0.96	0.95	0.95	0.95	0.95	0.95	0.94	B8	1.01	1.00	1.00	1.00	0.99	0.99	0.99	0.99	B8	1.01	1.01	1.01	1.00	1.00	1.00	1.00	0.99
B9	0.93	0.92	0.92	0.92	0.92	0.92	0.92	0.91	B9	0.98	0.98	0.98	0.97	0.97	0.97	0.97	0.96	B9	0.99	0.99	0.99	0.98	0.98	0.98	0.98	0.99
B10	1.10	1.10	1.10	1.10	1.10	1.10	1.10	1.09	B10	1.09	1.08	1.07	1.06	1.06	1.05	1.04	1.04	B10	1.16	1.14	1.12	1.12	1.12	1.11	1.11	1.11
B11	1.02	1.02	1.02	1.02	1.02	1.01	1.01	1.01	B11	1.02	1.02	1.02	1.01	1.01	1.01	1.00	1.00	B11	1.04	1.03	1.03	1.02	1.01	1.01	1.00	1.00
B12	1.02	1.01	1.01	1.01	1.01	1.01	1.00	1.00	B12	1.02	1.02	1.01	1.01	1.01	1.00	1.00	1.00	B12	1.04	1.04	1.03	1.02	1.01	1.01	1.01	1.00
B13	1.01	1.01	1.00	1.00	1.00	1.00	1.00	1.00	B13	1.02	1.01	1.01	1.00	1.00	1.00	1.00	1.00	B13	1.04	1.04	1.06	1.08	1.09	1.07	1.06	1.06
B14	1.43	1.44	1.45	1.44	1.41	1.39	1.38	1.37	B14	1.32	1.31	1.29	1.28	1.27	1.26	1.25	1.25	B14	1.39	1.37	1.35	1.35	1.30	1.26	1.24	1.23
B15	1.23	1.23	1.22	1.21	1.20	1.19	1.18	1.17	B15	1.21	1.20	1.19	1.19	1.18	1.17	1.17	1.16	B15	1.18	1.18	1.17	1.14	1.12	1.11	1.09	1.08

Figure 6: Finite Element Analysis (FEA) inspired local block secant gain (G_i) heatmap across context lengths from 2K to 256K tokens for all 16 blocks. In NoPE, structural yield initiates at Block 6 and accelerates at Block 7 ($G = 1.376$ at 256K). Polar (ATMA) acts as an internal energy-dissipating damper, maintaining flat secant gains across extreme context extensions.

G_i (local operator stiffness):

$$G_i = \frac{\|f_i(x + \delta x) - f_i(x)\|_{\text{rms}}}{\|\delta x\|_{\text{rms}}}, \quad \text{where } \|\delta x\|_{\text{rms}} = 0.02 \cdot \|x\|_{\text{rms}} \text{ under isotropic } d \sim \mathcal{N}(0, I). \quad (26)$$

462 The maximum sampled secant gain G_{max} provides a tight empirical lower bound on the local
463 block Lipschitz constant.

464 5.6.2 Yield Locus and Architectural Weak Point

465 The FEA heatmap visualization in Figure 6 displays the raw 16×8 secant gain data across
466 sequence lengths and localizes three distinct structural behaviors:

- 467 1. Polar (Internal Energy Damper): Clean NLL remains flat (1.275 at 128K $\rightarrow$ 1.292
468 at 256K). Maximum block secant gain declines smoothly from 1.427 to 1.369, and
469 attention-projection RMS remains stable. Unbounded participation growth n_{eff} is
470 absorbed internally by radial unit normalization ($\|c_i\|_2 \leq 1$) and bounded $\tanh$
471 magnitude ($\text{mag}_i \in [0, 1)$), acting like an internal energy-dissipating damper that
472 prevents structural yield across 256K contexts.
- 473 2. RoPE (Representational Attenuation): NLL degrades smoothly (1.950 to 2.846).
474 Block secant gain falls alongside shrinking MLP and residual RMS signals, signaling
475 positional frequency domain mismatch and representational attenuation rather than
476 noise amplification.
3. NoPE (Unstable Yield Locus): NLL explodes to 10.766 at 256K. As displayed in
Figure 6, the FEA probe localizes the structural yield locus / weak point to a
specific depth chain:

$$\underbrace{\text{Block 6 Attn}}_{\text{Initial Stress Concentration}} \longrightarrow \underbrace{\text{Block 7 MLP}}_{\text{Shear Amplification (} G = 1.376 \text{)}} \longrightarrow \underbrace{\text{Block 10 Attn}}_{\text{Structural Buckling}} \quad (27)$$

477 Block 6 Attention acts as the initial stress concentration node; Block 7 MLP accel-
478 erates the response gain (rising from 1.018 at 2K to 1.376 at 256K); and Block 10
479 Attention experiences catastrophic activation explosion down the residual backbone,
480 causing NLL collapse.

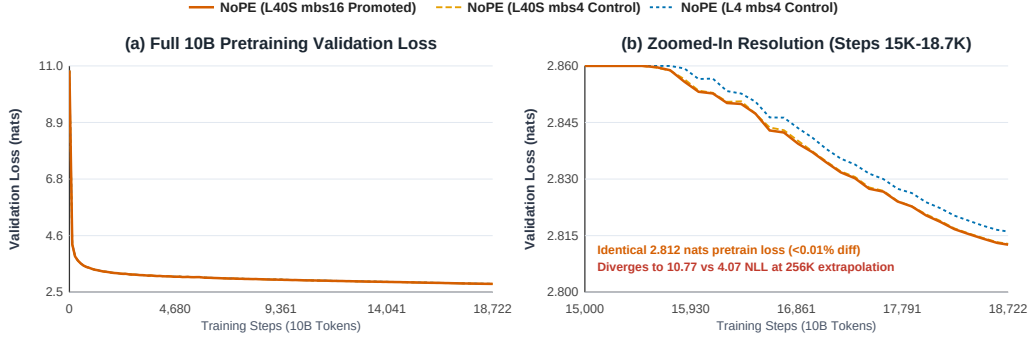


Figure 7: Pretraining validation loss trajectories across 18,722 steps (10B tokens) for the three NoPE checkpoints. Panel (a) shows the full pretraining loss trajectory. Panel (b) zooms in on the final 3,722 pretraining steps (0.001-nat resolution), demonstrating that pretraining curves are virtually identical (2.8126 vs 2.8128 vs 2.8160 nats, $< 0.01\%$ relative difference), despite their extreme 256K extrapolation divergence (10.77 vs 9.90 vs 4.07 nats NLL).

5.7 Environment Sensitivity and Pretraining Indistinguishability

A striking empirical finding in our diagnostic logs is that short-context pretraining convergence alone provides zero warning signal for long-context extrapolation collapse.

As illustrated in Figure 7, during 2K-context pretraining across 10B tokens, all three NoPE variants achieve virtually identical, statistically indistinguishable validation loss trajectories:

- NoPE (L4 mbs4 Control): 1.9011 nats pretraining loss at 2K (2.8161 nats validation loss at step 18,722).
- NoPE (L40S mbs16 Promoted): 1.8900 nats pretraining loss at 2K (2.8126 nats validation loss at step 18,722).
- NoPE (L40S mbs4 Control): 1.8804 nats pretraining loss at 2K (2.8128 nats validation loss at step 18,722).

Even when zoomed in to 0.001-nat resolution (Figure 7b), the pretraining loss curves are completely overlapping ($< 0.01\%$ relative difference).

However, when subjected to 256K context extrapolation in the FEA stress probe (Appendix G), their extrapolation behaviors diverge radically:

- NoPE (L4 mbs4 Control): Extrapolates stably at 256K context (4.0668 nats NLL), with Block 7 secant gain remaining flat at 1.014.
- NoPE (L40S mbs16 Promoted): Suffers catastrophic activation explosion at 256K context (10.7659 nats NLL), with Block 7 secant gain surging to 1.376.
- NoPE (L40S mbs4 Control): Suffers near-identical activation explosion at 256K context (9.9048 nats NLL), with Block 7 secant gain surging to 1.401.

5.7.1 The Empirical Diagnostic Journey: From L4 to L40S

The discovery of this environment sensitivity followed a systematic diagnostic sequence:

1. Initial 1B Sweeps and 10B Control on L4: We executed all Stage I 1B-token ablation sweeps on NVIDIA L4 (24GB) GPUs. We then trained our first Stage II 10B NoPE control model on L4 using microbatch size 4 (mbs4), observing solid 2K $\rightarrow$ 256K extrapolation (4.07 nats NLL at 256K).
2. Transition to L40S and Unexpected Extrapolation Divergence: To accelerate the remaining 10B comparisons, we transitioned our primary training infrastructure

to NVIDIA L40S (48GB) GPUs and trained the promoted NoPE checkpoint using microbatch size 16 (mbs16). Surprisingly, while 2K pretraining validation loss curves were virtually identical (2.8126 vs 2.8161 nats, < 0.01% difference; Figure 7), the L40S mbs16 checkpoint suffered catastrophic 256K extrapolation collapse (10.77 nats NLL).

3. Systematic Diagnostic Elimination: We systematically investigated potential causal factors:

- Evaluation Environment / PyTorch Version? Re-evaluating L4 and L40S checkpoints under PyTorch 2.12 versus 2.13 yielded identical scores, ruling out evaluation software differences.
- Microbatch Accumulation Size? Retraining NoPE on L40S with mbs4 produced near-identical 256K activation explosion (9.90 nats NLL), proving microbatch size alone was not the cause.
- Deterministic Stream Fixtures? Running deterministic synthetic stream fixtures (scripts/diagnose_training_artifact.py) across PyTorch versions and microbatch sizes showed no gradient or loss divergence at 2K.

We concluded that reproducing the exact L4 mbs4 pretraining weight trajectory on L40S involves low-level non-deterministic CUDA kernel reduction and floating-point accumulation order across GPU architectures.

5.7.2 Fairness of the Main Comparison and a Methodological Audit Call

Crucially, all five headline Stage II 10B models (NoPE, RoPE, Polar, Atma-Raven-Titans, and Raven Native) were trained in the exact same L40S hardware and software environment. If GPU-level kernel reduction order introduced an implicit numerical regularization challenge during pretraining, Polar was subjected to the exact same environment and successfully absorbed context extensions out to 256K (1.825 BPB vs 10.77 NoPE), establishing a strictly controlled and fair comparison.

These empirical findings carry major methodological implications for the broader long-context research community. In many long-context extrapolation studies, an architecture’s failure or success at extended contexts (8K $\rightarrow$ 256K) is attributed entirely to its high-level mathematical design (e.g., positional embeddings, norm types, attention scaling). Our controlled diagnostics demonstrate that low-level hardware execution order alone can completely alter long-context extrapolation trajectories without leaving any detectable trace in short-context pretraining loss curves. We therefore issue an explicit audit call to long-context researchers: claims regarding architecture-driven extrapolation failures or successes must be audited across multiple hardware device classes and microbatch execution environments before attributing results solely to architectural mechanics.

5.8 Implementation Notes

Polar’s participation ratio requires streaming both the softmax denominator L and squared denominator Q^2 . When the online maximum changes by $\alpha = \exp(M_{\text{old}} - M_{\text{new}})$, the fused Triton kernel rescales L and the value sum by α , but Q^2 by α^2 , recovering $n_{\text{eff}} = L^2/Q^2$ without materializing the $T \times T$ score matrix. The paged decode kernel groups the four query heads sharing each KV head, and the gated-delta kernel updates recurrent state in place. These kernels make the 256K evaluations feasible, but Table 5 deliberately reports end-to-end backend-inclusive measurements rather than attributing all performance to one kernel.

6 Conclusion

This work frames long-context sequence modeling not as a search for a single universal architecture, but as an empirical Pareto frontier balancing soft retrieval fidelity, reasoning capacity, long-document likelihood stability, and inference compute efficiency. Across matched 10B-token training runs, Polar Attention establishes a compelling Pareto balance

by preserving soft-attentional retrieval fidelity (16.3% real-text accuracy at 64K vs 0.0% for NoPE/RoPE) and long-document stability ($1.26\times$ BPB inflation at 256K) at a modest 1.5% short-context expressivity trade-off. Meanwhile, stateful Raven models offer $O(1)$ recurrent memory that excels at multi-step reasoning (46% BABILong at 256K) and $7.2\times$ faster decode throughput, and standard Softmax retains a minor edge on short 2K completion tasks.

Crucially, our diagnostic stress analysis uncovers a striking empirical warning: short-context pretraining convergence is completely blind to long-context extrapolation collapse, with identical 2K pretraining validation loss trajectories masking radical 256K context divergences across hardware device classes (L4 vs L40S) due to CUDA kernel reduction order. We issue a call to action for the field to adopt multi-hardware environment auditing protocols alongside architectural reporting. By releasing our complete repository artifact, configs, logs, and diagnostic suites, we aim to establish a transparent baseline for auditable, multi-objective long-context sequence modeling research.

References

- Arshia Afzal, Aviv Bick, Eric P. Xing, Volkan Cevher, and Albert Gu. Raven: High-recall sequence modeling with sparse memory routing. arXiv preprint arXiv:2607.25357, 2026. URL <https://arxiv.org/abs/2607.25357>.
- Habibullah Akbar. Weak-sigreg: Covariance regularization for stable deep learning. arXiv preprint arXiv:2603.05924, 2026.
- Zeyuan Allen-Zhu. Physics of language models: Part 4.1, architecture design and the magic of canon layers. arXiv preprint arXiv:2512.17351, 2025.
- Randall Balestriero and Yann LeCun. Lejepa: Provable and scalable self-supervised learning without the heuristics. arXiv preprint arXiv:2511.08544, 2025.
- Federico Barbero, Andrea Banino, Steven Kapturowski, Dharshan Kumaran, Joao G. M. Araujo, Alex Vitvitskyi, Razvan Pascanu, and Petar Velickovic. Transformers need glasses! information over-squashing in language tasks. In Advances in Neural Information Processing Systems, 2024.
- Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. arXiv preprint arXiv:2501.00663, 2025.
- Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. arXiv preprint arXiv:2004.05150, 2020.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. arXiv preprint arXiv:2312.00752, 2023.
- Ramin Hasani et al. Lfm2 technical report. arXiv preprint arXiv:2511.23404, 2025.
- Xingyue Huang, Xueying Ding, Mingxuan Ju, Yozen Liu, Neil Shah, and Tong Zhao. Threshold differential attention for sink-free, ultra-sparse, and non-dispersive language modeling. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics, 2026. URL <https://arxiv.org/abs/2601.12145>.
- Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Ivan Rodkin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev. Babilong: Testing the limits of llms with long context reasoning-in-a-haystack. In Advances in Neural Information Processing Systems Datasets and Benchmarks Track, 2024. URL <https://arxiv.org/abs/2406.10149>.
- Ken M. Nakanishi. Scalable-softmax is superior for attention. arXiv preprint arXiv:2501.19399, 2025.
- Ofir Press, Noah A Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. arXiv preprint arXiv:2108.12409, 2021.

- 607 Jianlin Su et al. Roformer: Enhanced transformer with rotary position embedding. arXiv
608 preprint arXiv:2104.09864, 2021.
- 609 Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez,
610 Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Advances in Neural
611 Information Processing Systems, volume 30, pp. 5998–6008. Curran Associates, Inc., 2017.
- 612 Pavlo Vasylenko, Hugo Pitorro, Andre F. T. Martins, and Marcos Treviso. Long-context
613 generalization with sparse attention. In International Conference on Learning Represen-
614 tations, 2026.
- 615 Petar Velickovic, Christos Perivolaropoulos, Federico Barbero, and Razvan Pascanu. Soft-
616 max is not enough (for sharp size generalisation). In International Conference on Machine
617 Learning, 2025.
- 618 Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2
619 with delta rule. arXiv preprint arXiv:2412.06464, 2024a.
- 620 Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear
621 transformers with the delta rule over sequence length. arXiv preprint arXiv:2406.06484,
622 2024b.
- 623 Songlin Yang et al. Flash linear attention. [https://github.com/fla-org/](https://github.com/fla-org/flash-linear-attention)
624 [flash-linear-attention](https://github.com/fla-org/flash-linear-attention), 2024c.

A Theoretical Framing

This appendix analyzes the role of the extreme-value-corrected null sink and the bounded ATMA mixer. The statements are sufficient-condition results: they establish noise-odds and norm bounds under explicit distributional and bounded-state assumptions, but they do not establish distributional invariance or replace the empirical ablation evidence in Section 4.

A.1 Extreme-Value Null Floor

Fix one query and let X_j denote the score of an irrelevant key after the same normalization used by Polar Attention. The TDA analysis motivates the following noise model.

Assumption 1 (Sub-Gaussian noise scores). For irrelevant keys $j \in \mathcal{N}_n$, $|\mathcal{N}_n| \leq n$, the scores are mean-zero and v^2 -sub-Gaussian for some $v > 0$:

$$\mathbb{E} \exp(tX_j) \leq \exp\left(\frac{v^2 t^2}{2}\right) \quad \text{for all } t \in \mathbb{R}. \quad (28)$$

Polar uses a learned floor of the form

$$\nu_n = b + s\sqrt{\log(n+1)}, \quad b \geq 0, \quad s \geq 0, \quad (29)$$

where b corresponds to `null_base` and s to the positive learned null-slope. The implementation does not constrain the learned base to remain nonnegative, so the results below apply to trained heads for which $b \geq 0$; alternatively, a nonnegative parameterization of `null_base` would enforce this condition. The next proposition is the direct analogue of the TDA survivor-count bound, with the hard threshold replaced by the Polar null floor.

Proposition 1 (Noise exceedances above the Polar null floor). Under Assumption 1, the expected number of irrelevant keys whose score exceeds ν_n is

$$\mathbb{E} \left[\sum_{j \in \mathcal{N}_n} \mathbf{1}\{X_j > \nu_n\} \right] \leq n \exp\left(-\frac{\nu_n^2}{2v^2}\right). \quad (30)$$

In particular, if $s \geq \sqrt{2}v$, this expectation is $O(1)$; if $s > \sqrt{2}v$, it decays polynomially in n .

Proof. For each irrelevant key, sub-Gaussian concentration gives $\Pr[X_j > \nu_n] \leq \exp(-\nu_n^2/(2v^2))$. Summing this tail probability over at most n irrelevant keys proves the bound. If $b \geq 0$, then $\nu_n^2 \geq s^2 \log(n+1)$, so the right-hand side is at most $n(n+1)^{-s^2/(2v^2)}$, which is bounded when $s^2/(2v^2) \geq 1$ and decays when the inequality is strict. $\square$

A.2 Soft Null-Sink Odds

Unlike TDA, Polar does not discard sub-threshold keys. It appends the null floor as a softmax competitor. For $n \geq 2$, let

$$\theta_n = 1 + a \log n, \quad a \geq 0, \quad (31)$$

be the length temperature and define the real-noise-to-null odds

$$R_n = \sum_{j \in \mathcal{N}_n} \exp(\theta_n(X_j - \nu_n)). \quad (32)$$

Then R_n is exactly the ratio between the unnormalized softmax mass assigned to irrelevant real keys and the unnormalized mass assigned to the null sink, restricted to noise keys.

Proposition 2 (Null odds vanish under an extreme-value margin). Suppose that, for some constants c and s with $s > c > \sqrt{2}v$, the learned null floor has $\nu_n = s\sqrt{\log(n+1)}$ up to a nonnegative offset. With probability tending to one,

$$R_n \leq n \exp\left(-(1 + a \log n)(s - c)\sqrt{\log n}\right). \quad (33)$$

If $a > 0$, this upper bound converges to zero faster than any inverse polynomial in n .

Proof. By the same union bound as Proposition 1,

$$\Pr \left[\max_{j \in \mathcal{N}_n} X_j > c\sqrt{\log(n+1)} \right] \leq n(n+1)^{-c^2/(2v^2)} \rightarrow 0, \quad (34)$$

because $c > \sqrt{2}v$. On the complementary event, every irrelevant score is at least $(s - c)\sqrt{\log n}$ below the null floor, hence each summand in R_n is at most $\exp(-(1 + a \log n)(s - c)\sqrt{\log n})$. Summing over at most n noise keys gives the displayed bound. For $a > 0$, the logarithm of the bound is $\log n - a(s - c)(\log n)^{3/2} + O(\sqrt{\log n})$, which tends to $-\infty$ faster than $-\rho \log n$ for any fixed ρ . $\square$

This result is the theoretical bridge from TDA to Polar: TDA enforces the extreme-value threshold by hard sparsification, while Polar uses the same scaling to make the null sink win the softmax competition against pure background noise.

A.3 Bounded Direction and Magnitude Channels

Proposition 3 (Polar readout is radially normalized and bounded). For the implementation-level normalization with $\varepsilon > 0$, define

$$s_i = \sum_j w_{ij}v_j + w_{iN}v_{\text{null}}, \quad c_i = \frac{s_i}{\max(\|s_i\|_2, \varepsilon)}. \quad (35)$$

Then $\|c_i\|_2 \leq 1$, with equality whenever $\|s_i\|_2 \geq \varepsilon$. The magnitude channel

$$\text{mag}_i = \tanh(\beta \log(1 + m_{\text{eff},i})) \quad (36)$$

lies in $[0, 1)$ for $\beta \geq 0$ and all $m_{\text{eff},i} \geq 0$. If the normalized real-key distribution is exactly uniform on a support of m keys, its participation ratio is $n_{\text{eff}} = m$.

Proof. The norm bound follows directly from the denominator. The magnitude bound follows from $\log(1 + m_{\text{eff},i}) \geq 0$ and $\tanh(x) \in [0, 1)$ for finite $x \geq 0$. For m equally weighted real keys, $\hat{w}_{ij} = 1/m$ on the support, so $n_{\text{eff}} = 1 / \sum_{j=1}^m (1/m)^2 = m$. $\square$

Thus the content channel removes radial scale while the magnitude channel exposes a bounded summary of effective match count. This is a channel-level decomposition, not a claim that c_i is generally independent of count: changing the real-key mixture or its balance with a nonzero learned v_{null} can rotate the direction.

A.4 Bounded Recurrent Memory State

The recurrent memory branch is also bounded under standard normalized-key and contractive-gate conditions. Consider the gated-delta update used by ATMA:

$$M_t = \gamma_t M_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top. \quad (37)$$

Assumption 2 (Contractive memory update). For all t , $\|q_t\|_2 = \|k_t\|_2 = 1$, $\|v_t\|_2 \leq V$, $0 \leq \beta_t \leq 1$, and $0 \leq \gamma_t \leq \bar{\gamma} < 1$.

Proposition 4 (Gated-delta state norm is length-bounded). Under Assumption 2,

$$\|M_t\|_F \leq \bar{\gamma}^t \|M_0\|_F + \frac{V}{1 - \bar{\gamma}}. \quad (38)$$

Proof. Since $\|k_t\|_2 = 1$ and $0 \leq \beta_t \leq 1$, the matrix $I - \beta_t k_t k_t^\top$ has spectral norm at most one. Also $\|\beta_t v_t k_t^\top\|_F \leq V$. Therefore $\|M_t\|_F \leq \bar{\gamma} \|M_{t-1}\|_F + V$. Unrolling this scalar recurrence proves the bound. $\square$

Corollary 1 (ATMA mixer is bounded under fixed weights). Under the conditions above, and for fixed projection operator norms, the per-layer ATMA mixing output

$$\mathbf{out} = \mathbf{content} + \mathbf{count} + \mathbf{memory} \quad (39)$$

is bounded independently of sequence length.

Proof. The content channel is a projection of head directions with norm at most one multiplied by sigmoid gates, the count channel is a projection of magnitudes in $[0, 1)$, and the memory readout is bounded by Proposition 4 and the fixed readout projection norms. The sum of finitely many bounded terms is bounded independently of n . $\square$

A.5 Scope of the Machine-Checked Artifact

The accompanying Lean artifact machine-checks the finite-sum, exponential-monotonicity, bounded-channel composition, and invariant-set induction steps used above. The sub-Gaussian tail inequality, the high-probability extreme-value margin, and the one-step matrix contraction are supplied to Lean as premises rather than derived from measure-theoretic random variables and the implemented gated-delta recurrence. We therefore describe it as a machine-checked proof skeleton, not an end-to-end formal verification of the ATMA method. The learned conditions $b \geq 0$ and $s > c > \sqrt{2}v$ are sufficient conditions and are not asserted to hold for every trained head without an empirical parameter-and-score audit.

B Regularization Modes

The ablation grid varies a representation regularizer applied to the model’s signature stream. For all non-baseline settings we use the same sweep weight, $\alpha_{\text{sig}} = 0.01$, and train with

$$\mathcal{L} = (1 - \alpha_{\text{sig}})\mathcal{L}_{\text{LM}} + \alpha_{\text{sig}}\mathcal{L}_{\text{reg}} + \lambda_{\text{dist}}\mathcal{L}_{\text{dist}}. \quad (40)$$

The weak and strong modes follow the two SIGReg families introduced by Weak-SIGReg (Akbar, 2026) and LeJEPA’s strong SIGReg objective (Balestrierio & LeCun, 2025); the discrete and zipfian modes are local variants that interpolate those ideas with stronger geometric priors.

Mode	Meaning in the sweep
baseline	No signature-stream regularization is used; equivalently, $\alpha_{\text{sig}} = 0$.
weak	Weak SIGReg: center a random sketch of the token representations and match its covariance to the identity, $\ \text{Cov}(Sx) - I\ _F$. This encourages decorrelated, unit-variance features at low cost.
strong	Strong SIGReg: project representations onto random one-dimensional directions and match their empirical characteristic functions to the standard Gaussian characteristic function over a fixed grid of frequencies.
discrete	Local variant: first normalize each representation to the sphere of radius $\sqrt{d}$, then apply the weak covariance-matching objective. This preserves the whitening pressure while encouraging fixed-norm, discrete-code-like directions.
zipfian	Local variant: normalize directions for an orthogonality penalty, then match the sorted representation magnitudes to a Zipf profile r^{-1} . This biases the signature stream toward heavy-tailed feature usage rather than uniform energy allocation.

Table 6: Regularization modes used in the ablation grid. The weak and strong modes are taken from SIGReg-style covariance and distribution matching; the discrete and zipfian modes are improvised variants used to probe stronger geometric structure.

C Polar Attention Algorithms

This appendix gives code-level implementations for Polar Attention. Code 1 shows the materialized PyTorch oracle (`polar_reduce` in `model/blocks.py`), which is used for numer-

693 ical verification. Code 2 shows the core of the FlashAttention-style Triton forward kernel
694 (kernel/polar_triton.py), which streams keys in blocks and maintains the same sufficient
695 statistics without materializing the $T \times T$ score matrix.

696 C.1 Naive Materialized PyTorch Reference

Listing 1: Materialized PyTorch reference for Polar Attention.

```

697 def polar_reduce(sigma, v, n_keys, *, v_null, null_base,
698                 null_slope_raw, len_gain_raw, mag_beta_raw,
699                 eps=1e-6):
700     """Reference oracle. sigma: (B,H,Tq,Tk), v: (B,H,Tk,dk)."""
701     out_dtype = v.dtype
702     cd = torch.float32 if v.dtype in (torch.float16, torch.bfloat16) else v.dtype
703     B, H, Tq, Tk = sigma.shape
704     dk = v.shape[-1]
705     sigma, v = sigma.to(cd), v.to(cd)
706
707     n = n_keys.to(cd).clamp(min=1.0)
708     temp = 1.0 + F.softplus(len_gain_raw.to(cd)).view(1, H, 1, 1) \
709         * torch.log(n).view(1, 1, Tq, 1)
710     null = null_base.to(cd).view(1, H, 1, 1) \
711         + F.softplus(null_slope_raw.to(cd)).view(1, H, 1, 1) \
712         * torch.sqrt(torch.log(n + 1.0)).view(1, 1, Tq, 1)
713
714     # Scale finite entries, then re-apply the mask. This avoids NaNs from
715     # differentiating through temp * (-inf).
716     masked = torch.isneginf(sigma)
717     sigma_safe = torch.where(masked, torch.zeros_like(sigma), sigma)
718     real = (sigma_safe * temp).masked_fill(masked, float("-inf"))
719
720     logits = torch.cat([real, null.expand(B, H, Tq, 1) * temp], dim=-1)
721     w = torch.softmax(logits, dim=-1)
722     w_r = w[..., :-1]
723     w_null = w[..., -1:]
724
725     # Direction channel: convex value mix plus learned null vector, projected
726     # radially so the content channel has bounded norm.
727     s = torch.matmul(w_r, v) + w_null * v_null.to(cd).view(1, H, 1, dk)
728     c = F.normalize(s, p=2, dim=-1, eps=eps)
729
730     # Magnitude channel: inverse Simpson participation ratio over real keys,
731     # gated by non-null confidence and squashed to [0, 1).
732     denom = w_r.sum(-1, keepdim=True).clamp_min(eps)
733     w_hat = w_r / denom
734     n_eff = 1.0 / w_hat.square().sum(-1).clamp_min(eps)
735     m_eff = n_eff * (1.0 - w_null.squeeze(-1))
736     beta = F.softplus(mag_beta_raw.to(cd)).view(1, H, 1)
737     mag = torch.tanh(beta * torch.log1p(m_eff))
738
739     return c.to(out_dtype), mag.to(out_dtype)
740

```

742 C.2 Flash-Style Triton Forward Kernel

743 Code 2 lists the Triton streaming forward pass. Note that the Triton magnitude computa-
 744 tion uses the hyperbolic identity $2\sigma(2x) - 1 \equiv \tanh(x)$, where beta is pre-transformed via
 745 softplus($\cdot$) upon loading, ensuring exact algebraic equivalence with the PyTorch reference
 746 implementation (`torch.tanh(beta * torch.log1p(m_eff))`).

Listing 2: Core Triton streaming forward pass. The full kernel includes pointer arithmetic, launch metadata, and backward kernels.

```

747 @triton.jit
748 def _polar_fwd_kernel_core(q, K, V, VNULL, SPG, NULLBASE, SPS, BETA,
749                          n_i, h, Tk, scale, eps,
750                          BLOCK_N: tl.constexpr, DK: tl.constexpr,
751                          WINDOW: tl.constexpr):
752     # Per-query length temperature and extreme-value-corrected null floor.
753     if WINDOW > 0:
754         n_count = tl.maximum(tl.minimum(n_i, float(WINDOW)), 1.0)
755     else:
756         n_count = tl.maximum(n_i, 1.0)
757     temp = 1.0 + tl.load(SPG + h).to(tl.float32) * tl.log(n_count)
758     nullv = tl.load(NULLBASE + h).to(tl.float32) \
759         + tl.load(SPS + h).to(tl.float32) * tl.sqrt(tl.log(n_count + 1.0))
760     beta = tl.load(BETA + h).to(tl.float32)
761
762     # Online softmax state. Q2 is the extra accumulator needed for the
763     # participation ratio; it rescales by alpha**2 under max-shift updates.
764     m_i = tl.full([BLOCK_M], -1e38, tl.float32)
765     l_i = tl.zeros([BLOCK_M], tl.float32)
766     q2_i = tl.zeros([BLOCK_M], tl.float32)
767     acc = tl.zeros([BLOCK_M, DK], tl.float32)
768
769     for start_n in range(0, Tk, BLOCK_N):
770         offs_n = start_n + tl.arange(0, BLOCK_N)
771         k = load_key_block(K, offs_n) # (BLOCK_N, DK), native dtype
772         v = load_value_block(V, offs_n) # (BLOCK_N, DK), native dtype
773
774         sig = tl.dot(q, tl.trans(k)) * scale
775         a = sig * temp[:, None]
776         valid = offs_n[None, :] < n_i[:, None]
777         if WINDOW > 0:
778             valid = valid & (offs_n[None, :] >= (n_i[:, None] - WINDOW))
779         a = tl.where(valid, a, -1e38).to(tl.float32)
780
781         m_new = tl.maximum(m_i, tl.max(a, 1))
782         alpha = tl.exp(m_i - m_new)
783         p = tl.exp(a - m_new[:, None])
784         p = tl.where(valid, p, 0.0)
785
786         l_i = l_i * alpha + tl.sum(p, 1)
787         q2_i = q2_i * alpha * alpha + tl.sum(p * p, 1)
788         acc = acc * alpha[:, None] + tl.dot(p, v)
789         m_i = m_new
790
791     # Fold the virtual null sink as one additional softmax entry.
792     a_null = temp * nullv
793     m_new = tl.maximum(m_i, a_null)
794     alpha = tl.exp(m_i - m_new)
795     l_i = l_i * alpha
796     q2_i = q2_i * alpha * alpha
797     acc = acc * alpha[:, None]
798     m_i = m_new
799     p_null = tl.exp(a_null - m_i)
800     Z = l_i + p_null
801
802     v_null = tl.load(VNULL + h * DK + tl.arange(0, DK)).to(tl.float32)
803     s = acc + p_null[:, None] * v_null[None, :]
804     c = s / tl.maximum(tl.sqrt(tl.sum(s * s, 1)), eps)[:, None]
805
806     n_eff = l_i * l_i / tl.maximum(q2_i, eps)
807     m_eff = n_eff * (l_i / tl.maximum(Z, eps))
808     mag = 2.0 * tl.sigmoid(2.0 * beta * tl.log(1.0 + m_eff)) - 1.0
809
810     store_direction(c)
811     store_magnitude(mag)
812     save_for_backward(m_i, l_i, q2_i, s)
813

```

D Complete Ablation Sweep Results

We present the full numerical results of our 120-run factorial ablation sweep. The table lists the attention type, signature-stream regularization mode, distractor status, memory branch status, sliding-window status, training validation loss, clean-document NLL and junk-stream NLL at multipliers $1\times$ and $32\times$ (2K and 64K tokens), needle-in-a-haystack per-digit accuracy at 2K and 64K tokens, and training Model Flops Utilization (MFU).

Table 7: Complete Results of the 120-Run Ablation Sweep. All models are 370M parameters, trained on FineWeb-Edu for 1B tokens ($seq_len = 2048$). Evaluated from 2K to 64K context length. Clean NLL is measured on coherent FinePDFs documents; junk NLL is measured on the concatenated validation stream. NLL is in nats/token (lower is better), Needle is greedy per-digit accuracy in % (higher is better), and MFU is training model flops utilization in %.

Type	Reg	Dist	Mem	Win	Val	Clean2K	Clean64K	Junk2K	Junk64K	Ndl2K	Ndl64K	MFU
POLAR	baseline	Off	Off	Off	3.323	2.83	3.60	3.27	5.68	96.3	0.0	40.1
POLAR	baseline	Off	Off	On	3.343	2.83	2.66	3.30	4.74	67.5	0.0	39.6
POLAR	baseline	Off	On	Off	3.169	2.70	1.96	3.11	3.13	91.3	92.5	35.6
POLAR	baseline	Off	On	On	3.174	2.71	2.01	3.12	3.13	51.3	30.0	35.6
POLAR	baseline	On	Off	Off	3.332	2.81	5.56	3.28	6.54	85.0	0.0	33.3
POLAR	baseline	On	Off	On	3.361	2.85	2.89	3.35	4.97	91.3	2.5	34.5
POLAR	baseline	On	On	Off	3.178	2.72	1.98	3.12	3.14	73.8	58.8	31.8
POLAR	baseline	On	On	On	3.182	2.73	1.97	3.12	3.14	56.3	21.3	31.2
POLAR	weak	Off	Off	Off	3.333	2.82	2.86	3.28	5.35	88.8	0.0	39.0
POLAR	weak	Off	Off	On	3.348	2.90	3.76	3.38	5.24	93.8	0.0	37.9
POLAR	weak	Off	On	Off	3.179	2.74	1.95	3.12	3.14	70.0	51.3	36.2
POLAR	weak	Off	On	On	3.187	2.74	1.99	3.13	3.15	41.3	41.3	36.2
POLAR	weak	On	Off	Off	3.335	2.82	3.47	3.28	5.56	91.3	0.0	32.8
POLAR	weak	On	Off	On	3.353	2.82	2.58	3.32	4.81	96.3	1.3	32.8
POLAR	weak	On	On	Off	3.178	2.74	1.95	3.12	3.14	33.8	32.5	30.7
POLAR	weak	On	On	On	3.183	2.74	1.94	3.13	3.15	81.3	78.8	30.7
POLAR	strong	Off	Off	Off	3.338	2.85	3.79	3.28	5.83	98.8	1.3	37.3

Continued on next page

Table 7: Complete Results of the 120-Run Ablation Sweep (continued).

Type	Reg	Dist	Mem	Win	Val	Clean2K	Clean64K	Junk2K	Junk64K	Ndl2K	Ndl64K	MFU
POLAR	strong	Off	Off	On	3.360	2.85	2.99	3.32	5.01	91.3	0.0	37.9
POLAR	strong	Off	On	Off	3.172	2.72	1.96	3.11	3.13	80.0	52.5	35.8
POLAR	strong	Off	On	On	3.181	2.72	1.94	3.12	3.15	76.3	68.8	34.6
POLAR	strong	On	Off	Off	3.337	2.87	3.22	3.28	5.52	83.8	1.3	32.8
POLAR	strong	On	Off	On	3.347	2.82	2.64	3.30	4.88	97.5	16.3	32.2
POLAR	strong	On	On	Off	3.179	2.73	1.94	3.12	3.14	86.3	83.8	30.9
POLAR	strong	On	On	On	3.182	2.72	1.94	3.12	3.14	25.0	23.8	30.9
POLAR	discrete	Off	Off	Off	3.338	2.82	2.80	3.29	5.12	93.8	2.5	37.9
POLAR	discrete	Off	Off	On	3.349	2.87	2.90	3.33	5.25	93.8	6.3	37.9
POLAR	discrete	Off	On	Off	3.184	2.75	2.03	3.12	3.14	76.3	73.8	36.4
POLAR	discrete	Off	On	On	3.178	2.71	1.92	3.12	3.15	31.3	18.8	35.8
POLAR	discrete	On	Off	Off	3.333	2.80	4.13	3.28	6.12	97.5	0.0	32.6
POLAR	discrete	On	Off	On	3.351	2.87	2.72	3.33	4.89	85.0	2.5	33.3
POLAR	discrete	On	On	Off	3.189	2.74	1.96	3.13	3.15	87.5	53.8	31.8
POLAR	discrete	On	On	On	3.192	2.74	2.07	3.13	3.15	21.3	26.3	31.7
POLAR	zipfian	Off	Off	Off	3.338	2.85	4.83	3.28	6.55	96.3	11.3	37.9
POLAR	zipfian	Off	Off	On	3.350	2.83	2.58	3.32	4.63	88.8	3.8	39.4
POLAR	zipfian	Off	On	Off	3.171	2.70	1.92	3.11	3.13	52.5	63.8	36.2
POLAR	zipfian	Off	On	On	3.180	2.75	1.95	3.12	3.15	46.3	40.0	35.2
POLAR	zipfian	On	Off	Off	3.340	2.83	3.77	3.28	5.64	98.8	1.3	33.8
POLAR	zipfian	On	Off	On	3.352	2.85	3.13	3.36	5.23	90.0	0.0	33.5
POLAR	zipfian	On	On	Off	3.177	2.72	1.94	3.12	3.14	91.3	67.5	31.8
POLAR	zipfian	On	On	On	3.192	2.74	2.00	3.13	3.15	40.0	26.3	31.8
NOPE	baseline	Off	Off	Off	3.224	2.76	3.71	3.16	6.68	97.5	1.3	41.5
NOPE	baseline	Off	Off	On	3.219	2.75	2.76	3.17	5.49	92.5	10.0	36.5
NOPE	baseline	Off	On	Off	3.140	2.66	2.34	3.09	3.23	97.5	16.3	36.8
NOPE	baseline	Off	On	On	3.150	2.70	2.14	3.09	3.24	81.3	0.0	34.2
NOPE	baseline	On	Off	Off	3.209	2.74	2.99	3.15	6.13	90.0	18.8	32.5
NOPE	baseline	On	Off	On	3.212	2.77	2.90	3.17	5.65	81.3	0.0	30.3
NOPE	baseline	On	On	Off	3.149	2.67	2.35	3.10	3.28	80.0	16.3	30.0

Continued on next page

Table 7: Complete Results of the 120-Run Ablation Sweep (continued).

Type	Reg	Dist	Mem	Win	Val	Clean2K	Clean64K	Junk2K	Junk64K	Ndl2K	Ndl64K	MFU
NOPE	baseline	On	On	On	3.147	2.68	2.09	3.09	3.23	88.8	21.3	28.2
NOPE	weak	Off	Off	Off	3.214	2.77	3.10	3.16	6.34	95.0	0.0	40.9
NOPE	weak	Off	Off	On	3.222	2.75	3.05	3.19	5.81	81.3	0.0	36.1
NOPE	weak	Off	On	Off	3.147	2.68	2.07	3.09	3.19	88.8	21.3	37.5
NOPE	weak	Off	On	On	3.142	2.68	2.12	3.08	3.23	82.5	0.0	33.7
NOPE	weak	On	Off	Off	3.215	2.75	4.87	3.16	12.48	95.0	5.0	32.0
NOPE	weak	On	Off	On	3.221	2.73	2.63	3.18	4.84	81.3	0.0	29.3
NOPE	weak	On	On	Off	3.147	2.67	2.48	3.09	3.23	97.5	1.3	30.8
NOPE	weak	On	On	On	3.148	2.68	2.14	3.09	3.24	93.8	1.3	28.4
NOPE	strong	Off	Off	Off	3.231	2.74	4.88	3.18	11.85	90.0	2.5	38.4
NOPE	strong	Off	Off	On	3.214	2.72	2.68	3.17	5.28	82.5	0.0	36.4
NOPE	strong	Off	On	Off	3.144	2.68	2.37	3.09	3.24	96.3	6.3	35.7
NOPE	strong	Off	On	On	3.150	2.70	2.10	3.10	3.25	93.8	7.5	33.8
NOPE	strong	On	Off	Off	3.207	2.72	3.13	3.15	6.20	73.8	0.0	31.4
NOPE	strong	On	Off	On	3.214	2.74	2.49	3.17	4.36	86.3	8.8	28.9
NOPE	strong	On	On	Off	3.146	2.67	2.24	3.09	3.23	83.8	17.5	29.1
NOPE	strong	On	On	On	3.147	2.68	2.16	3.09	3.28	96.3	0.0	27.5
NOPE	discrete	Off	Off	Off	3.202	2.75	3.33	3.15	6.23	85.0	1.3	38.8
NOPE	discrete	Off	Off	On	3.234	2.79	3.11	3.19	5.47	70.0	18.8	37.0
NOPE	discrete	Off	On	Off	3.145	2.68	2.14	3.09	3.27	98.8	16.3	36.0
NOPE	discrete	Off	On	On	3.145	2.68	2.10	3.09	3.25	95.0	3.8	34.3
NOPE	discrete	On	Off	Off	3.212	2.88	3.04	3.16	6.88	42.5	0.0	31.2
NOPE	discrete	On	Off	On	3.211	2.78	2.73	3.16	5.28	82.5	1.3	29.1
NOPE	discrete	On	On	Off	3.145	2.68	2.27	3.08	3.26	91.3	11.3	30.3
NOPE	discrete	On	On	On	3.149	2.68	2.17	3.08	3.32	83.8	0.0	28.6
NOPE	zipfian	Off	Off	Off	3.209	2.71	3.18	3.15	6.40	90.0	0.0	40.8
NOPE	zipfian	Off	Off	On	3.219	2.75	2.71	3.17	5.44	77.5	1.3	36.1
NOPE	zipfian	Off	On	Off	3.141	2.67	2.55	3.08	3.38	96.3	0.0	38.0
NOPE	zipfian	Off	On	On	3.150	2.69	2.11	3.09	3.25	90.0	3.8	33.7
NOPE	zipfian	On	Off	Off	3.207	2.77	3.32	3.15	7.09	82.5	0.0	31.3

Continued on next page

Table 7: Complete Results of the 120-Run Ablation Sweep (continued).

Type	Reg	Dist	Mem	Win	Val	Clean2K	Clean64K	Junk2K	Junk64K	Ndl2K	Ndl64K	MFU
NOPE	zipfian	On	Off	On	3.227	2.79	2.91	3.18	5.94	93.8	11.3	30.2
NOPE	zipfian	On	On	Off	3.146	2.68	2.39	3.10	3.25	96.3	21.3	29.6
NOPE	zipfian	On	On	On	3.148	2.68	2.17	3.09	3.28	90.0	6.3	28.7
ROPE	baseline	Off	Off	Off	3.159	2.85	3.36	3.09	5.21	42.5	0.0	40.4
ROPE	baseline	Off	Off	On	3.163	2.86	3.79	3.17	5.86	15.0	0.0	37.2
ROPE	baseline	Off	On	Off	3.145	2.81	2.86	3.09	3.57	73.8	0.0	37.1
ROPE	baseline	Off	On	On	3.151	2.80	2.95	3.12	3.72	13.8	0.0	34.4
ROPE	baseline	On	Off	Off	3.163	3.00	3.47	3.10	5.46	38.8	0.0	33.2
ROPE	baseline	On	Off	On	3.167	2.88	4.06	3.16	5.92	0.0	0.0	30.2
ROPE	baseline	On	On	Off	3.153	2.73	2.45	3.09	3.57	75.0	0.0	31.4
ROPE	baseline	On	On	On	3.154	2.83	2.86	3.12	3.72	32.5	0.0	29.4
ROPE	weak	Off	Off	Off	3.158	2.88	3.46	3.10	5.36	46.3	0.0	41.4
ROPE	weak	Off	Off	On	3.163	2.84	3.96	3.18	5.93	12.5	0.0	37.8
ROPE	weak	Off	On	Off	3.149	2.71	2.46	3.09	3.56	37.5	0.0	36.5
ROPE	weak	Off	On	On	3.156	2.85	2.77	3.12	3.60	23.8	0.0	34.9
ROPE	weak	On	Off	Off	3.155	2.83	3.49	3.10	5.27	38.8	0.0	32.3
ROPE	weak	On	Off	On	3.157	3.05	4.12	3.18	5.86	8.8	0.0	30.5
ROPE	weak	On	On	Off	3.149	2.77	2.76	3.09	3.56	57.5	0.0	31.1
ROPE	weak	On	On	On	3.149	2.77	3.13	3.12	3.80	17.5	0.0	28.0
ROPE	strong	Off	Off	Off	3.165	2.78	3.34	3.10	5.27	30.0	0.0	39.0
ROPE	strong	Off	Off	On	3.169	2.90	3.68	3.19	5.80	31.3	0.0	36.3
ROPE	strong	Off	On	Off	3.158	2.85	2.63	3.10	3.50	51.3	0.0	35.9
ROPE	strong	Off	On	On	3.162	2.86	3.08	3.14	3.76	10.0	0.0	33.2
ROPE	strong	On	Off	Off	3.161	2.85	3.58	3.10	5.28	47.5	0.0	31.1
ROPE	strong	On	Off	On	3.165	2.85	4.03	3.19	5.90	2.5	0.0	29.2
ROPE	strong	On	On	Off	3.155	2.80	2.70	3.10	3.62	61.3	0.0	29.3
ROPE	strong	On	On	On	3.162	2.85	3.08	3.13	3.74	2.5	0.0	28.4
ROPE	discrete	Off	Off	Off	3.165	2.91	3.43	3.11	5.42	22.5	0.0	39.7
ROPE	discrete	Off	Off	On	3.158	2.84	4.05	3.15	5.90	3.8	0.0	36.9
ROPE	discrete	Off	On	Off	3.159	2.83	2.67	3.11	3.56	47.5	0.0	36.3

Continued on next page

Table 7: Complete Results of the 120-Run Ablation Sweep (continued).

Type	Reg	Dist	Mem	Win	Val	Clean2K	Clean64K	Junk2K	Junk64K	Ndl2K	Ndl64K	MFU
ROPE	discrete	Off	On	On	3.161	2.79	2.71	3.13	3.74	41.3	0.0	33.7
ROPE	discrete	On	Off	Off	3.171	2.79	3.45	3.12	5.36	51.3	0.0	31.5
ROPE	discrete	On	Off	On	3.164	2.80	3.74	3.16	5.86	21.3	0.0	30.0
ROPE	discrete	On	On	Off	3.162	2.74	2.47	3.11	3.53	50.0	0.0	30.9
ROPE	discrete	On	On	On	3.154	2.79	2.68	3.11	3.67	18.8	0.0	28.9
ROPE	zipfian	Off	Off	Off	3.163	2.82	3.25	3.10	5.22	62.5	0.0	39.8
ROPE	zipfian	Off	Off	On	3.161	2.94	3.94	3.18	5.81	16.3	0.0	37.1
ROPE	zipfian	Off	On	Off	3.149	2.78	2.70	3.09	3.63	46.3	0.0	36.7
ROPE	zipfian	Off	On	On	3.151	2.80	2.63	3.11	3.68	2.5	0.0	33.9
ROPE	zipfian	On	Off	Off	3.161	2.78	3.38	3.10	5.32	47.5	0.0	32.6
ROPE	zipfian	On	Off	On	3.160	3.01	4.11	3.17	5.81	17.5	0.0	30.2
ROPE	zipfian	On	On	Off	3.149	2.73	2.39	3.09	3.55	62.5	0.0	29.8
ROPE	zipfian	On	On	On	3.152	2.79	2.99	3.12	3.78	26.3	0.0	28.5

819 E Pretrained Open-Weight Baseline Benchmark Results

820 Table 8 reports zero-shot evaluations across 10 pretrained open-weight models ranging from
 821 230M to 752M parameters without length adaptation. Clean NLL and Junk NLL are re-
 822 ported in nats/token; Needle is greedy per-digit accuracy in percent over 16 five-digit induc-
 823 tion trials.

Model	Params	Clean 2K (nats)	Clean 64K (nats)	Junk 2K (nats)	Junk 64K (nats)	Needle 2K (%)	Needle 64K (%)
Qwen/Qwen3.5-0.8B-Base	752M	1.94	1.33	2.61	2.61	100.0	100.0
Qwen/Qwen3-0.6B-Base	596M	1.95	1.40	2.60	2.65	100.0	95.0
Qwen/Qwen2.5-0.5B	494M	2.03	1.53	2.63	2.68	100.0	38.1
google/gemma-3-270m	268M	2.48	1.90	3.23	3.16	96.2	24.4
ibm-granite/granite-4.0-h-350m-base	340M	2.22	2.06	2.66	2.82	75.0	54.4
ibm-granite/granite-4.0-350m-base	352M	2.52	3.02	2.68	4.00	40.6	8.8
tiiuae/Falcon-H1-0.5B-Base	521M	1.99	2.54	2.32	3.49	92.5	19.4
HuggingFaceTB/SmolLM2-360M	362M	1.99	6.04	2.46	7.04	100.0	8.1
LiquidAI/LFM2.5-230M-Base	230M	3.01	6.69	3.74	7.66	38.1	19.4
LiquidAI/LFM2.5-350M-Base	354M	3.18	6.63	4.06	7.73	45.6	46.3

Table 8: Zero-shot performance of 10 pretrained open-weight baselines evaluated on our benchmark harness without context adaptation.

F Serving Latency and Roofline Throughput Matrix

Table 9 presents the exact prefill throughput, prefill latency, decode latency, and decode throughput scaling across sequence lengths from 2K to 128K tokens on one NVIDIA L40S GPU.

Model	Metric	2K	4K	8K	16K	32K	64K	128K
Polar	Prefill Latency (s)	0.131	0.031	0.051	0.101	0.248	0.601	1.619
	Prefill Throughput (tok/s)	15,640	133,884	160,145	161,551	132,091	109,050	80,959
	Decode Latency (ms/tok)	2.27	2.49	2.88	3.70	5.66	9.22	16.30
	Decode Throughput (tok/s)	441.2	402.3	346.6	270.1	176.8	108.5	61.3
NoPE	Prefill Latency (s)	0.157	0.028	0.049	0.097	0.230	0.576	1.473
	Prefill Throughput (tok/s)	13,070	146,568	166,504	168,510	142,319	113,810	89,013
	Decode Latency (ms/tok)	2.22	2.45	2.83	3.68	5.62	9.22	16.40
	Decode Throughput (tok/s)	450.1	408.8	353.0	271.7	177.8	108.4	61.0
RoPE	Prefill Latency (s)	0.160	0.028	0.049	0.096	0.232	0.566	1.472
	Prefill Throughput (tok/s)	12,761	147,089	168,287	170,936	141,249	115,786	89,015
	Decode Latency (ms/tok)	2.34	2.53	2.95	3.79	5.73	9.33	16.51
	Decode Throughput (tok/s)	427.4	395.3	338.7	263.7	174.5	107.2	60.6
Atma-Raven-Titans	Prefill Latency (s)	0.198	0.041	0.059	0.111	0.247	0.499	0.999
	Prefill Throughput (tok/s)	10,334	100,989	137,903	147,434	132,916	131,464	131,195
	Decode Latency (ms/tok)	2.24	2.25	2.24	2.24	2.26	2.25	2.27
	Decode Throughput (tok/s)	446.6	445.1	446.0	447.1	442.6	444.8	440.3
Raven Native	Prefill Latency (s)	0.214	0.054	0.070	0.139	0.320	0.672	1.357
	Prefill Throughput (tok/s)	9,558	75,738	116,378	117,605	102,354	97,527	96,559
	Decode Latency (ms/tok)	3.26	3.25	3.24	3.24	3.25	3.25	3.27
	Decode Throughput (tok/s)	306.4	307.5	308.2	308.3	307.6	307.5	306.1

Table 9: Detailed prefill and decode latency and throughput scaling across context lengths from 2K to 128K tokens on one NVIDIA L40S GPU (batch size 1, 32 generated tokens).

Systems Scale Benchmarks against Production vLLM (Qwen3.5-9B). Table 10 provides a direct systems-scale performance comparison between ATMA’s Triton engine (stress harness at 9.2B parameter scale) and production vLLM (v0.25.0) serving Qwen3.5-9B on the same NVIDIA L40S GPU in BF16 precision.

Workload Phase	Batch / Workload Config	ATMA Engine	vLLM (Qwen3.5-9B)	ATMA Speedup
Prefill	Homogeneous $B = 8, T = 512$ (4,096 tok)	11,947 tok/s	12,772 tok/s	0.94×
	Heterogeneous Short-Heavy (816 tok)	5,752 tok/s	8,002 tok/s	0.72×
	Heterogeneous Mixed (4,384 tok)	12,779 tok/s	11,996 tok/s	1.07×
	Heterogeneous Long-Tail (8,192 tok)	12,251 tok/s	11,721 tok/s	1.05×
Decode (Context 512)	Batch size 128	2,939 tok/s	2,247 tok/s	1.31×
	Batch size 256	4,501 tok/s	2,921 tok/s	1.54×
	Batch size 512	5,919 tok/s	2,558 tok/s	2.31×
	Heterogeneous $B = 128$ (Context 64–1536)	2,950 tok/s	2,403 tok/s	1.23×

Table 10: Systems-scale throughput comparison between ATMA’s custom Triton engine (9.2B parameter stress shape) and production vLLM 0.25.0 serving Qwen3.5-9B on one NVIDIA L40S GPU (BF16, greedy sampling).

G Structural Stress Analysis and Block Secant Gain Data

Table 11 reports maximum sampled local block secant gains $G_{\max}$ across sequence lengths from 2K to 256K context tokens for Polar, RoPE, and all three testable NoPE configurations in the repository (L40S mbs16 promoted, L40S mbs4 control, and L4 mbs4 control).

Block 14 exhibits the highest intrinsic secant gain across all architectures under short sequences. Both L40S NoPE checkpoints (mbs16 and mbs4) develop an explosive stress concentration at Block 7 ($G = 1.376$ and 1.401 at 256K), whereas the L4 NoPE checkpoint exhibits a milder Block 10 redistribution (1.233 at 256K).

Model Architecture	Length 2K	Length 16K	Length 65K	Length 256K
Polar (ATMA)	Block 14: 1.427	Block 14: 1.435	Block 14: 1.393	Block 14: 1.369
	Block 15: 1.234	Block 15: 1.211	Block 15: 1.191	Block 15: 1.174
	Block 10: 1.103	Block 10: 1.102	Block 10: 1.098	Block 6: 1.109
RoPE	Block 14: 1.325	Block 14: 1.277	Block 14: 1.259	Block 14: 1.249
	Block 15: 1.207	Block 15: 1.186	Block 15: 1.172	Block 15: 1.163
	Block 10: 1.093	Block 4: 1.070	Block 10: 1.053	Block 10: 1.040
NoPE (L40S mbs16 Promoted)	Block 14: 1.390	Block 14: 1.346	Block 14: 1.258	Block 7: 1.376
	Block 15: 1.183	Block 15: 1.144	Block 7: 1.204	Block 14: 1.232
	Block 10: 1.164	Block 10: 1.115	Block 10: 1.112	Block 10: 1.108
NoPE (L40S mbs4 Control)	Block 14: 1.276	Block 14: 1.282	Block 7: 1.304	Block 7: 1.401
	Block 10: 1.162	Block 10: 1.149	Block 10: 1.172	Block 10: 1.209
	Block 6: 1.057	Block 7: 1.112	Block 14: 1.089	Block 6: 1.121
NoPE (L4 mbs4 Control)	Block 14: 1.364	Block 14: 1.351	Block 14: 1.225	Block 10: 1.233
	Block 10: 1.128	Block 10: 1.173	Block 10: 1.208	Block 14: 1.213
	Block 6: 1.059	Block 6: 1.045	Block 6: 1.042	Block 7: 1.014

Table 11: Top three maximum sampled local block secant gains $G_{\max}$ across context length extension from 2K to 256K tokens for Polar, RoPE, and all three testable NoPE configurations.

840 H Diagnostic Harness Code Listing

841 Listing 3 provides the core Python implementation for tracking gradi-
 842 ent and parameter signatures across microbatch sizes and PyTorch versions
 843 (scripts/diagnose_training_artifact.py).

Listing 3: Core gradient and parameter vector signature tracking for accumulation divergence analysis.

```

844 def __is_critical(name: str) -> bool:
845     return (
846         ".attn.mem." in name
847         or any(token in name for token in (
848             "len_gain_raw", "null_base", "null_slope_raw", "mag_beta_raw",
849             "v_null", "mu_proj", ".attn.q.", ".attn.k.", ".attn.v.",
850             ".attn.canon_", ".attn.proj.",
851         ))
852     )
853
854 def __vector_payload(tensor: torch.Tensor) -> dict:
855     flat = tensor.detach().float().reshape(-1).cpu()
856     if flat.numel() <= CRITICAL_FULL_LIMIT:
857         values = flat.tolist()
858         mode = "full"
859     else:
860         count = min(CRITICAL_SAMPLE_SIZE, flat.numel())
861         indices = torch.linspace(0, flat.numel() - 1, steps=count, dtype=torch.float64).round().long()
862         values = flat[indices].tolist()
863         mode = "sample"
864     return {
865         "mode": mode,
866         "numel": flat.numel(),
867         "values": values,
868         "norm": torch.linalg.vector_norm(flat).item(),
869         "max_abs": flat.abs().max().item() if flat.numel() else 0.0,
870         "sum": flat.sum().item(),
871         "finite": bool(torch.isfinite(flat).all()),
872     }
873
874 def __tensor_signatures(model, gradients: bool) -> dict:
875     groups = {}
876     critical = {}
877     for name, parameter in model.named_parameters():
878         tensor = parameter.grad if gradients else parameter.data
879         if tensor is None:
880             continue
881         detached = tensor.detach().float()
882         group_name = __parameter_group(name)
883         accumulator = groups.setdefault(
884             group_name, {"sumsq": 0.0, "sum": 0.0, "max_abs": 0.0, "numel": 0, "finite": True}
885         )
886         accumulator["sumsq"] += detached.square().sum().item()
887         accumulator["sum"] += detached.sum().item()
888         accumulator["max_abs"] = max(accumulator["max_abs"], detached.abs().max().item())
889         accumulator["numel"] += detached.numel()
890         accumulator["finite"] = accumulator["finite"] and bool(torch.isfinite(detached).all())
891         if __is_critical(name):
892             critical[name] = __vector_payload(detached)
893     for accumulator in groups.values():
894         accumulator["norm"] = math.sqrt(accumulator.pop("sumsq"))
895     return {"groups": groups, "critical": critical}
896

```